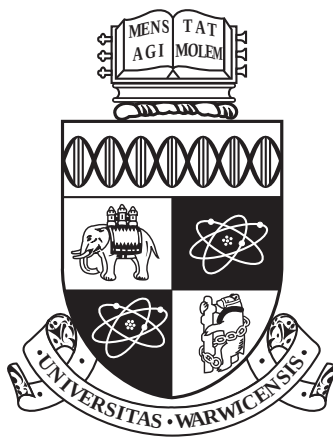


# Building Low-Poly Dioramas in Three.js

With a Focus on Autonomous Characters

**Natalie Robinson**  
**u2109897**



Supervisor: Dr. Rossella Suma  
**Department of Computer Science**  
University of Warwick  
2023 - 2024

## Abstract

For over 30 years computer graphics has revolutionised the way we interact with each other and the world around us. Professional 3D modelling software has advanced fields such as game design, film, and product manufacturing; as well as providing a creative outlet for many, as an artistic medium.

While many of this is freely available to the general public as open-source software, those who are unfamiliar with this domain of work often find themselves lost and discouraged due to the extensive time and knowledge required to create even the simplest of models.

This project focuses on the creation of a browser based software that allows the user to quickly construct a diorama of an interactive world, with no requirements for software or technical experience. The user is able to build a customised scene from a series of premade models and responsive characters, who will explore the created world along random paths. A collision detection system has been implemented using raycaster techniques to allow autonomous movement of the characters towards a randomly generated goal. This follows a path which avoids both static and dynamic objects that the user has placed into the scene.

This software is built on top of the Javascript API Three.js. This presents the opportunity to program alongside ready made function blocks which encapsulate the technical knowledge required to create a three dimensional scene. Less experienced users are presented with a fully graphical interface, to construct a world through a simple drag and drop system.

**Keywords:** JavaScript, Three.js, Computer Graphics, User Interactivity, Raycasting, Pathfinding, Object Avoidance, Collision Detection

# Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                        | <b>4</b>  |
| 1.1      | Project Motivation . . . . .               | 4         |
| 1.2      | Related Work . . . . .                     | 5         |
| 1.2.1    | User Interface . . . . .                   | 5         |
| 1.2.2    | Autonomous Characters . . . . .            | 8         |
| <b>2</b> | <b>Methodology</b>                         | <b>10</b> |
| 2.1      | Project Management Methodology . . . . .   | 10        |
| 2.2      | Software Development Methodology . . . . . | 10        |
| 2.3      | Ethics and Legality . . . . .              | 12        |
| <b>3</b> | <b>Design</b>                              | <b>13</b> |
| 3.1      | Requirements . . . . .                     | 13        |
| 3.1.1    | Core Objectives . . . . .                  | 13        |
| 3.1.2    | Extended Objectives . . . . .              | 15        |
| 3.1.3    | Non-Functional Requirements . . . . .      | 15        |
| 3.2      | GUI Design . . . . .                       | 16        |
| 3.2.1    | First Prototype . . . . .                  | 16        |
| 3.2.2    | Second Prototype . . . . .                 | 16        |
| 3.2.3    | Final Design . . . . .                     | 17        |
| 3.3      | Architecture Design . . . . .              | 17        |
| 3.3.1    | Class Relationships . . . . .              | 18        |
| 3.4      | Technologies . . . . .                     | 19        |
| 3.5      | Testing Strategy . . . . .                 | 19        |
| 3.5.1    | Functional Testing . . . . .               | 19        |
| 3.5.2    | Non-Functional Testing . . . . .           | 20        |
| 3.5.3    | Use Cases . . . . .                        | 20        |
| <b>4</b> | <b>Implementation</b>                      | <b>22</b> |
| 4.1      | World Creation . . . . .                   | 22        |
| 4.1.1    | Loading the Models . . . . .               | 22        |
| 4.1.2    | Exploring the World . . . . .              | 24        |
| 4.1.3    | Lighting the Scene . . . . .               | 25        |
| 4.2      | Character Movement . . . . .               | 28        |
| 4.2.1    | Static Collision Avoidance . . . . .       | 28        |
| 4.2.2    | Path Creation: Dynamic Splines . . . . .   | 28        |
| 4.2.3    | Path Creation: A* Algorithm . . . . .      | 31        |
| 4.2.4    | Generation of a Random Goal . . . . .      | 33        |
| 4.2.5    | Following the Path . . . . .               | 35        |
| 4.2.6    | Character Animations . . . . .             | 36        |
| 4.3      | Character Interactions . . . . .           | 38        |
| 4.3.1    | Dynamic Collision Avoidance . . . . .      | 38        |
| 4.3.2    | Approaches Considered . . . . .            | 38        |
| 4.4      | User Interactivity . . . . .               | 40        |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 4.4.1    | Adding Models to the World . . . . .          | 40        |
| 4.4.2    | Selecting Models . . . . .                    | 41        |
| 4.4.3    | Translating Model Positions . . . . .         | 41        |
| 4.4.4    | Clearing the World . . . . .                  | 42        |
| 4.4.5    | Generating Scenes . . . . .                   | 42        |
| 4.5      | Graphical User Interface . . . . .            | 43        |
| <b>5</b> | <b>Testing and Performance</b>                | <b>45</b> |
| 5.1      | User Testing . . . . .                        | 45        |
| 5.1.1    | Usability of Controls . . . . .               | 45        |
| 5.1.2    | Usability of the Interface . . . . .          | 46        |
| 5.1.3    | Recommended Features . . . . .                | 47        |
| 5.1.4    | Conclusions . . . . .                         | 47        |
| 5.2      | Performance Testing . . . . .                 | 48        |
| 5.2.1    | Launching the Application . . . . .           | 48        |
| 5.2.2    | Adding Objects to the World . . . . .         | 49        |
| 5.2.3    | Adding Characters to the World . . . . .      | 49        |
| 5.2.4    | Computing a Character's Path . . . . .        | 51        |
| 5.2.5    | Revising a Character's Path . . . . .         | 51        |
| 5.2.6    | Adding Multiple Characters . . . . .          | 52        |
| 5.3      | Stress Testing . . . . .                      | 52        |
| 5.3.1    | Maximum Models on a Block . . . . .           | 53        |
| 5.3.2    | Extreme Use Case: Adding Objects . . . . .    | 54        |
| 5.3.3    | Extreme Use Case: Adding Characters . . . . . | 55        |
| 5.4      | Optimisation Techniques . . . . .             | 55        |
| 5.5      | Future Performance Gains . . . . .            | 56        |
| 5.5.1    | Design Methodologies . . . . .                | 56        |
| <b>6</b> | <b>Project Management</b>                     | <b>57</b> |
| 6.1      | Reflection . . . . .                          | 57        |
| 6.2      | Unforeseen Issues . . . . .                   | 58        |
| <b>7</b> | <b>Evaluation</b>                             | <b>59</b> |
| 7.1      | Project Objectives . . . . .                  | 59        |
| 7.2      | Limitations . . . . .                         | 60        |
| 7.3      | Next Steps . . . . .                          | 60        |
| 7.3.1    | Interactivity Features . . . . .              | 60        |
| 7.3.2    | Technical Features . . . . .                  | 62        |
| 7.4      | Future Applications . . . . .                 | 62        |
| 7.4.1    | 3D Modelling and Visualisation . . . . .      | 62        |
| 7.4.2    | Game Creation . . . . .                       | 63        |
| 7.4.3    | Scene Creation . . . . .                      | 64        |
| 7.4.4    | Uses in Digital Art . . . . .                 | 64        |
| 7.5      | Conclusion . . . . .                          | 65        |
| <b>A</b> | <b>Dynamic Collision Logic</b>                | <b>70</b> |
| <b>B</b> | <b>Survey Responses</b>                       | <b>72</b> |
| <b>C</b> | <b>Original Project Schedule</b>              | <b>73</b> |

# Chapter 1

## Introduction

### 1.1 Project Motivation

Computer graphics has a wide range of applications: from modelling test products, creating virtual spaces, and developing the animation and film industry. While this medium is ever advancing, it is an area that is often overlooked by novices who may view it as inaccessible compared to other traditional design approaches. This is especially true for those with no experience in 3D modelling or using professional software. In a similar way, computer graphics, like many fields of computer science, is often seen as unapproachable to those with no technical or programming experience.

Professional modelling software, such as Blender, is often used in the production of animations. This design tool aids in the creation of 3D visualisations which can be explored with a freely controllable camera [1] that navigates along the axis planes. This is often used within the animation field to capture scenes from complex perspectives, and produce realistic backgrounds that can be seamlessly integrated alongside traditional hand-drawn techniques [2]. These may be professionally rendered and used as direct backgrounds for digital art and animated films; or are often used as a basis for scalable 2D vector drawings - combining the benefits of both modern and traditional techniques [3].

This is only one example of the wide variety of applications Blender provides, many more can be found within the user stores in their introductory manual [4].

However, this often comes with difficulties. Blender themselves state that their system “may seem daunting when first trying to grasp the basics” [1]. This is a perspective commonly agreed upon by many, and is applicable for most modelling software due to the extensive features to navigate and technical jargon involved. While they go on to discuss the plethora of online learning resources available to relieve this, we believe that this needed “hours” of dedication deters those hoping to step into the domain of 3D design.

This project aims to create an ease to the transition into systems such as Three.js. Three.js is an open-source graphical library in JavaScript, which provides high-level commands to simplify the use of the graphical rendering pipeline by abstracting over the WebGL API within web browsers [5]. This ensures that the created system is lightweight with little dependencies, and is readily available for all potential users.

This project is built as an extension of Three.js to provide further abstracted commands that minimise the graphical knowledge required. The main focus is on a fully graphical interface which welcomes newcomers, allowing them to easily construct their own customised scenes with no prior knowledge or skills required. These scenes are built from premade objects and characters, which have the capabilities of exploring the world along randomly generated paths.

The initial development of the project focuses on the use-case of creating a forest scene for characters to explore, the end result is shown in Figure 1.1.



Figure 1.1: An example forest diorama created from this project.

## 1.2 Related Work

In this section we compare similar systems and how they meet the main aims of our project, this includes the current focus of the project and future applications.

Our research phase has found that professional software often sacrifices usability in favour of complex features. This is, of course, highly sought out by professionals to produce high quality pieces of work; however, the domain becomes increasingly intimidating for a novice. This project is perfect for hobbyists who wish to dabble in computer design for one-off creations, and those who need an easy introduction before they decide that game design or graphical modelling is right for them.

### 1.2.1 User Interface

The first consideration of our project is the user interface, this is arguably the most important part of the system as dictates how the user will interact with the system and accomplish their goals. Design of the interface must take into consideration the potentially diverse user base of the software, and the varying levels knowledge that they will have. In ‘*Designing the User Interface* (2010)’, Shneiderman and Plaisant discuss effective strategies for human-computer interaction when designing for both novices and professionals. They state that the main aim for design is to create such an intuitive solution that “the interface almost disappears”. An application should provide innate focus on the user’s goal with minimal distractions from operating the interface. This is often achieved through repeated use of repeated familiar gestures, natural data layouts, and allowing direct manipulations through the screen [6].

Popular 3D modelling systems, such as Blender and Maya, often achieve this due to their fully graphical system; an example of Blender’s interface is given in Figure 1.2. This allows for precise adjustments to be made to an object, through direct selections of an objects vertices and immediate responses displayed on screen. This is a large contrast to the alternative of modelling within a programming environment such as WebGL or Three.js. The use of this introduces the need for extensive technical knowledge, focusing on meticulous adjustments along the graphical pipeline and precise positioning of each individual vertex.

A purely visual system removes the need for accurate syntax and allows for simple error detection and recovery. Rather than relying on the systems error messages, the user can follow the natural inverse action of their mistake, since all physical actions have clear results [6]. We also remove the mathematics necessary for creating a scene within WebGL, and the need for technicalities such as shader programs.

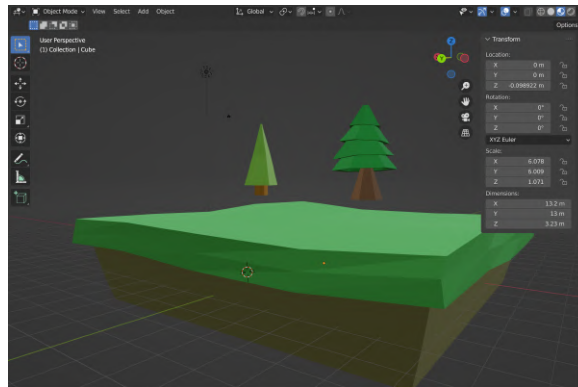


Figure 1.2: User Interface of Blender 4.1.0.0

Blender is often highly regarded as a widely versatile piece of graphical software, that is initially inviting to new users due to its popularity as a free open-source service. However, it also has the reputation of being difficult to learn – only aided by the extensive resources made available from the online community. It is a frequent re-occurrence that novices are discouraged after a few days of learning.

As part of the initial research phase for this project, unstructured interviews were conducted among students with experience in different modelling systems. A consensus was made that difficulties with these is often due to its excessive use of keyboard shortcuts and technical jargon, which hides features within menus and makes the search tool unproductive.

However, this view is not always shared. A practical study conducted by Snodgrass and Kontostathis compares the user experience when developing a 3D game within multiple environments. They found that Blender’s focus on the use of hotkeys and shortcuts allowed for a streamlined and efficient modelling process. While the alternative considered, Autodesk’s 3D Studio Max, lacks these features. Instead this software has a heavy reliance on hierarchical menus and taskbar icons, these can be seen in Figure 1.3. While the function of these icons are not always immediately clear, this is aided by labels that appear when hovered over by the user’s cursor. This allows the user to quickly learn how to identify each function. The usability study found that these graphical features help to ease navigation, compared to the reliance of hotkeys, but results in slower progress [7].

These conflicting views of Blender’s usability, are due to the differing prior knowledge and experience of the users involved. Those who are unfamiliar with 3d modelling are likely to initially be intimidated by the intricacies of Blender, however those with more experience will appreciate the smooth workflows and the wide functionality available.

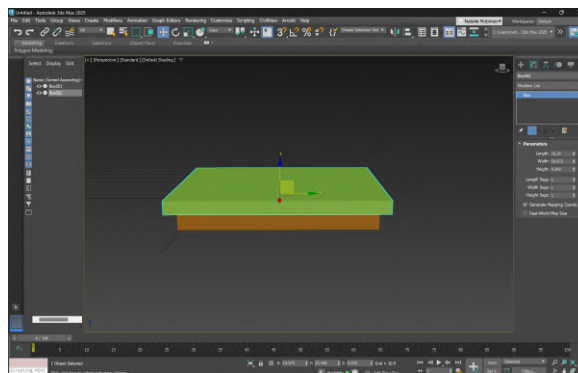


Figure 1.3: User Interface of AutoDesk 3Ds Max 2025

Shneiderman and Plaisant describe the importance of designing for novices and first-time users, accentuating the need for a restricted vocabulary and the ability to perform simple tasks through only a small number of actions. The use of common and consistent terms, rather than the technical jargon often favoured, allows professional software to remain approachable and increases retention over time.

While these features are essential for new users, those who are already experts will contrastingly favour shortcuts and accelerators, such as hotkeys and macros. This helps to achieve rapid response time when following complex procedures [6].

Designing a system which accompanies for this range of use, is achieved through a multi-layer approach. Shneiderman and Plaisant present the idea of a “training-wheels interface”. This allows the user to navigate through a minimal subset of actions, in order to build up key knowledge and experience necessary to progress onto more sophisticated tasks. Along with this, the interface expands to introduce more complex concepts once the user chooses to progress [6].

The development of this project aims to construct a software tool which provides a process similar to this - working as a stepping stone for any professional design software that may lack support for novices and first-time users.

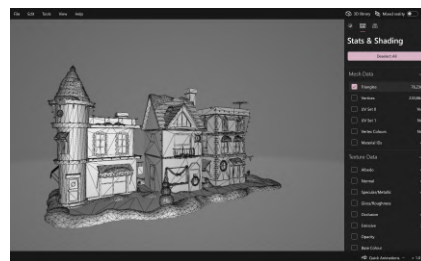
The Windows operating system has an in-built 3D model viewer, which presents a simple interface to visualise individual models. This can either be imported from the user’s documents, or chosen from a library of premade models. Currently, the focus is on the customisation of the camera and environment to allow the user to view their model within customised lighting conditions. As shown in Figure 1.4a, a ring allows the user to interactively manipulate the placement of the main light source to visualise how shadows are cast from the model, and use sliders to update the hue.

The user can easily navigate between pages which provide new functionalities. In Figure 1.4b we can see the mesh of the model, highlighting the edges and vertices used to construct it. This gives useful statistics such as the number of polygons and types of textures used to render the model - a feature that is particularly important for enhancing the performance of rendering models.

This application brings the user’s needs to focus, and works from similar motivation to our project. It allows for someone new to 3d modelling to easily view and experiment with premade models, and gives an opportunity for those with more experience to explore how their models behave in different environments. Our project will instead focus on the creation of scenes using these models, rather than the behaviour of them individually, but will take inspiration from the simple interface and navigation of this system.



(a) A menu allows for lighting and environment customisation.



(b) Viewing the model’s mesh and statistics

Figure 1.4: Windows 3D viewer and the scene “Snowy Village”.



### 1.2.2 Autonomous Characters

Shneiderman and Plaisant go on to make the claim that, after becoming familiar with a purely visual direct-manipulation interface, users tend to prefer this to a “complex syntactic notation to describe an essentially visual process” [6]. In the case of our project, this is comparable to studying the extensive Three.js library, when your aim is purely to design a fully graphical world.

A core aspect of our project is the ability to minimise this element of programming. For the focus of our project, a key technical asset allows the user to develop dynamic and interactive scenes without the need for scripting. This allows any characters added into the scene to automatically compute random paths that explore the created world. By developing this as a separate module, we introduce Non-playable Characters (NPCs) into Three.js. The function blocks can be reused in future projects. Future development would allow the user to modify these paths, to create customised background characters, and further interactions such as prompting dialogue in response to the user.

This brings Three.js closer to a game design engine such as Unity. This professional software works seamlessly with modelling tools such as Blender and Maya, being seen as a highly popular and useful tool for those with plenty of experience and dedication. Unity is often seen as essential software for 3D animation, world-building, and game development. An example of Unity’s interface is given in Figure 1.5.

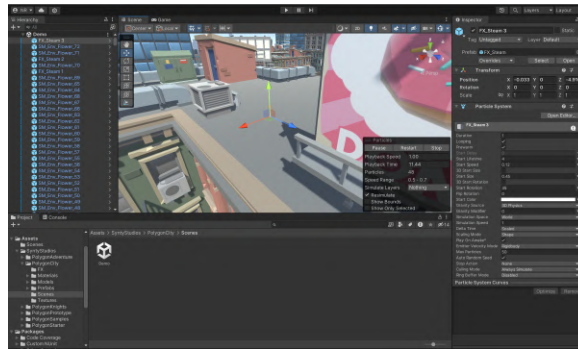


Figure 1.5: User Interface of Unity 2022.3.20f1 displaying the POLYGON Sampler Pack from SYNTY STUDIOS

Unity combats the intimidation that comes with these work areas, by offering extensive video tutorials and lesson plans through their Unity Learn platform. Here they provide “learning pathways” catered towards different levels of prior experience, to help familiarise the user with the basics of Unity’s functionality [8].

The addition of C# scripts alongside visual modifications, allows for extensive customisation of the created game, while streamlining the natural aspects of computer graphics.

In Unity, the movement of characters relies on the development of a Finite State Machine, with events prompting the character to move between predefined states [9]. A tutorial from Unity Learn provides an example of an NPC moving between its idle and walking state based on the generation of a random number. On this transition, character attributes are updated, such as the active animation [10], and a path to follow is computed based on a ‘Navigation Mesh’. This is a data structure describing the walkable surfaces within the world and allows pathfinding between objects through AI navigation [11]. Object avoidance is implemented through the use of a `NavMeshObstacle` which manipulates the `NavMesh` so that the occupied position is no longer walkable [12].

While not as sophisticated as Unity, the development of our project makes this technology available to those who do not already have sufficient programming expertise, or the time and dedication available to set aside countless hours to learn.

The autonomous navigation of characters has been extensively researched by many. A common approach to this problem is the implementation of both global and local navigation systems. The library VIRTUAL-ME has been constructed within Unity to model autonomous crowds of independent agents simulating human behaviour [13]. The authors have implemented a global navigation system by providing the agent with a learned map of the walkable areas - this is similar to the approach by Unity. The avoidance of dynamic objects, whose positions are not known prior to the simulation, are handled with a force-based approach. Each agent is able to perceive the environment and can therefore predict when a collision may occur. In this case, a force is applied to the character to change its direction of motion [13].

Our project follows a similar approach to this task breakdown. A character's navigation is aided through the monitoring of potential collisions with respect to both static and dynamic objects. The positioning of these static objects are dictated by the user and can be introduced at any time, therefore a graphical representation of the world is regularly updated to reflect the current state. The chosen implementations for this project are discussed in detail within Section 4.2.

# Chapter 2

## Methodology

This section discusses the main techniques used throughout the development of the project, and provides a reflection on the initial plans laid out in the Progress Report.

### 2.1 Project Management Methodology

This project loosely follows the standards and guidelines collected within ‘*A Guide to the Project Management Body of Knowledge*’ [14], alongside optimisations from the Lean business philosophy [15]. This allows us to minimise potential waste by focusing on the creation of the Minimum Viable Product given in the Project Specification. This is constructed from three key features: the creation of a graphical interface, the ability to build up a scene from preset models, and the introduction of characters who explore this scene.

The following of this framework is restricted due to the presence of a hypothetical client. Instead feedback is gained from students within our target demographics. This includes those who have experience with design technologies such as Blender and Unity, as well as those who have no knowledge of graphical design. The latter group has provided us with key feedback from an artistic and usability perspective.

This guide presents key process groups to follow: project initiation, planning, execution, project monitoring, and a closing stage [14]. Which are discussed throughout this report.

### 2.2 Software Development Methodology

The project execution stage begins with the acquirement of resources and encapsulates the main development phases, following an agile inspired methodology.

This consists of several iterative cycles which focus on the key objectives established within the planning stage, and have been scheduled based on estimated time frames. These have been given in Figures 2.1 and 2.2.

Inspired by the Lean principle that work is “pulled” between stages [15], a list of project features is kept and ordered by priority and work schedule. Each week, an objective is popped from the work queue and the next development iteration begins.

An embedded planning stage then takes place, which breaks the requirement down into sub-objectives that dictate how the feature will be implemented. This involves drafting key logic through pseudo code or flow diagram, and may involve a short period of research. These elements are implemented alongside a testing and evaluation cycle and, once all are validated, the feature is integrated alongside the rest of the system

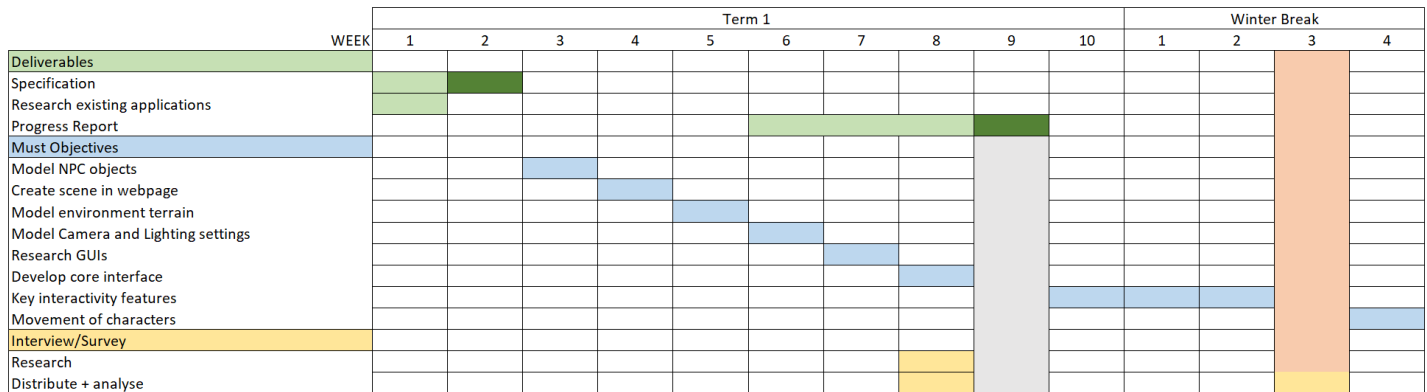


Figure 2.1: Gantt chart showing schedule followed with estimated time phases for Term 1

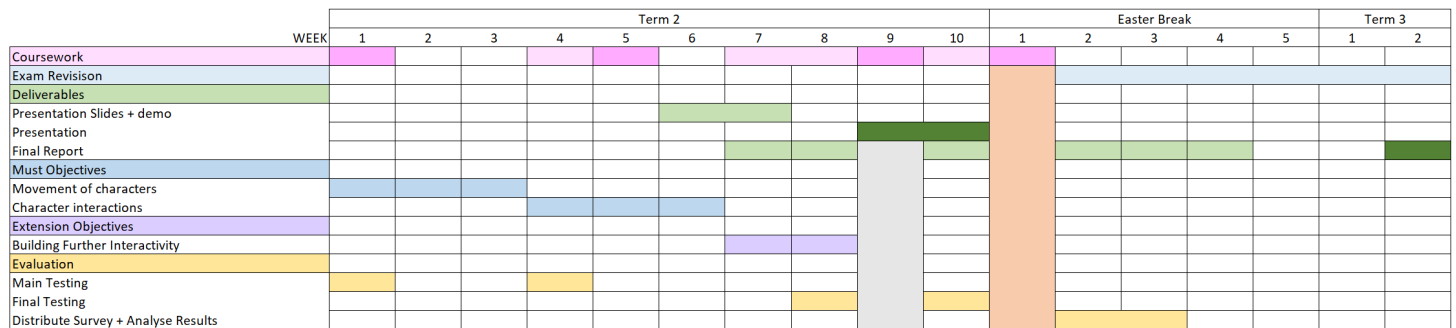


Figure 2.2: Gantt chart showing schedule followed with estimated time phases for Term 2

All stages are documented throughout; logging any decisions made, logic followed, and tests that are implemented. This allows us to monitor our progress through the project, ensuring that we are advancing as expected, and capture potential risks that may arise from schedule changes. Our time estimates for each development section can then be adjusted to accurately reflect the progress that is being made. Thorough documentation enables us to visualise the dependencies between each core objective, which is essential for splitting the system into separate modules for the user to import into their own code.

Organisations following an Agile based methodology have been found to be more efficient in terms of resource use, decision making, and overall product turnover; this is due to the innate ability to adapt in the face of unforeseen issues [16].

These gains are only achieved if agile is implemented effectively. This is not always the case for many organisations due to the needed paradigm shift from a more traditional waterfall development, this is since an agile approach requires a break down in team structure, to form smaller teams capable of rapid decision making, and consistent customer involvement. This change is often resisted [17].

However, in the case of our project, this does not pose as a risk due to our already small teams. Instead, alternate risks may arise from lack of personnel and an active customer. However this is mitigated through meetings with the project supervisor, which allows for effective decision-making due to the ability to discuss potential solutions, and gain feedback on current progress. The presence of a client's feedback stage is also captured through frequent user tests to ensure there is an engaging presentation of features aiding ease of use.

While these risks have been avoided to produce an effective methodology, the research phase is a key source of weakness. This focused hugely on the market research surrounding our project, however extending the research phase into Three.js and JavaScript would have cause for less research

and uncertainty when moving onto new development sections. Overall this would have resulted in more successful time management and a higher maintainability in the structure of the code. This is further discussed in Section 3.3.1.

## 2.3 Ethics and Legality

There is no need for ethical consent as we will not be gathering any personal data. All surveys will focus on providing feedback on the system and any potential features.

Graphical models have been taken from [quaternius.com](https://quaternius.com), included within ‘Ultimate Animated Animal Pack’ [18] and ‘Ultimate Modular Women Pack’ [19], these have been provided under the CC0 license.

The “Low-Poly Tent” model has been taken from [sketchfab.com](https://sketchfab.com) and is licensed under Creative Commons Attribution. Appropriate credits are given in the documentation.

The textures used for environment mapping have been taken from [hdri-haven.com](https://hdri-haven.com), where the assets are available in the public domain [20], or from [polyhaven.com](https://polyhaven.com). From the latter site, HDRI maps are available under the CC0 license, and do not require attribution with freedom of distribution [21].

# Chapter 3

## Design

This section covers the design of our project, providing a summary of the main goals considered when approaching the creation of this system, and an overview of the software’s logical structure.

### 3.1 Requirements

The main requirements for this project have been gathered within the Project Specification to construct our minimum-viable-product. This section provides a summary of the project’s key features, alongside a selection of extended interactivity features. These expand the customisability of the scene, as well as the user’s overall experience.

#### 3.1.1 Core Objectives

This project focuses on the use case of building a self-contained world from premade objects. This includes the creation of an intuitive, fully graphical system, with an extension of creating an abstraction to Three.js, encapsulating the technical knowledge necessary.

The core requirements have been decided through the construction of the use case diagram shown in Figure 3.1, this maps out how an average user will expect to interact with the system, including the key features that were highlighted within the market research phase of the project. Catering mainly to novices and first time users, the system will focus on the key features necessary for an untrained user to construct a scene.

The essential features have been listed in order of priority:

- C1 A fully graphical interface that allows the user to introduce pre-built models to the scene through a drag and drop system.
  - (a) Must include at least 2 characters and 1 scenery object.
  - (b) Should include 4 characters and 2 scenery objects.
- C2 The user can customise the placements of premade models to build up a 3D scene.
- C3 Characters explore the scene following random paths that:
  - (a) Must not leave the world boundary.
  - (b) Should avoid collisions with static scenery objects.
  - (c) Should resolve collisions amongst themselves.
  - (d) Could have basic animations on movement.
  - (e) Could have basic interactions animations when characters paths meet.
  - (f) Could have each character type moving at its own varying speed.

- C4 The world must be lighted according to a set “sun”. The user can choose between through preset lighting modes that reflect different times of day.
- C5 A graphical menu allows the user to choose between scenes and toggle relevant information such as character movement and speeds.
- C6 The user can freely pan the camera to explore the scene, this should include unrestricted zoom and rotation.
- C7 Includes an intuitive function to encapsulate all that is needed to set-up a scene in Three.js.
- C8 All commands must run alongside existing Three.js functions.

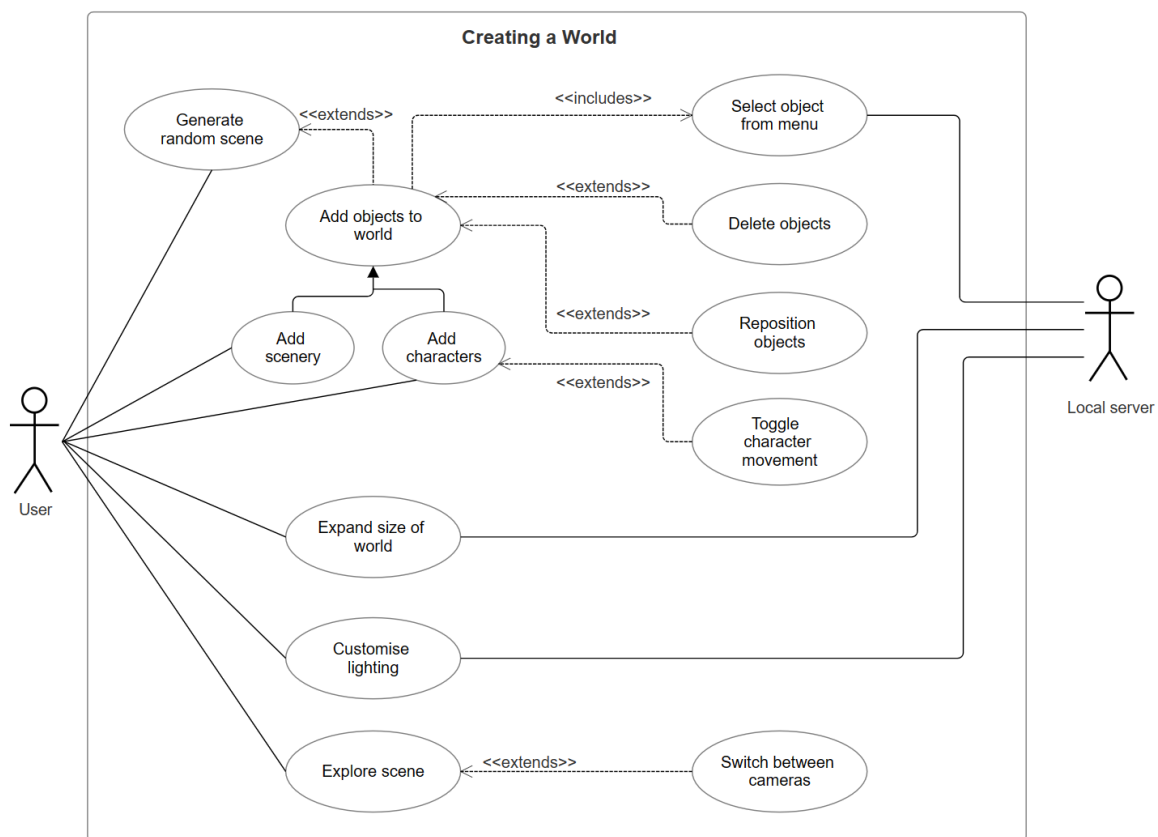


Figure 3.1: A Use Case Diagram visualising how a user will interact with the system. A local server is used to host the application, and handle all resource requests for assets such as the object models and textures.

### 3.1.2 Extended Objectives

These features have the lowest priority and will only be pulled into development once all essential features have been implemented and tested thoroughly. At this point, there will be a focus on increasing the interactivity of the system, with additional technical features left for future development phases.

These have been listed in order of priority:

- X1 A side-panel menu displays preview icons of each available model.
- X2 A minimalist button for cycling through lighting settings.
- X3 Present a secondary perspective from a top view camera, with a grid overlay, to aid in precise placement of objects.
- X4 Further customisation of the placement of objects, including rotation, duplication and deletion.
- X5 Shortcut option to clear all objects from the scene, must include a confirmation from the user.
- X6 Additional options to further customise the lighting settings, rather than being limited to the default options. This would include a sliding bar to edit the intensity of the main and ambient light sources, along with a graphical colour picker to change the hues.
- X7 Option to save and load world files.

### 3.1.3 Non-Functional Requirements

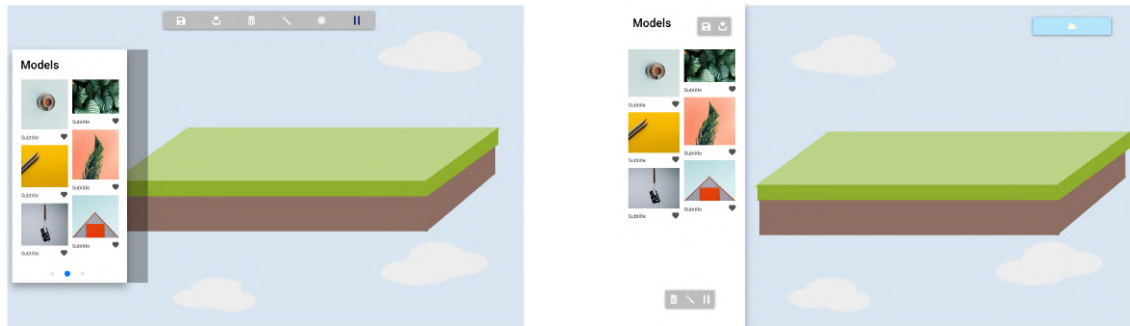
Since the key motivation for our project is accessibility, it is essential that the page remains responsive and simple to use. These requirements focus on the performance of the system, and highlight the need for responses to appear instantaneous for an average use case.

- R1 The application must load within 2 seconds when launched within a Chrome browser, this includes the time to render all elements onto the screen and become responsive to a users interactions.
- R2 Once the user selects an object from the graphical menu, this must be requested from the local server, loaded, and rendered into the scene within 0.1 seconds when under normal conditions.
- R3 The system must be capable of rendering 20 models into the world, with 90% of the frames being rendered within 16.6ms. This gives an average framerate of around 60fps, which will ensure smooth character animations [22].
- R4 When reaching the limits of the systems workload, the browser must respond to user requests within 3 seconds, with 90% of the frames being rendered within 33.3ms. This gives an average framerate of around 30fps [22].



## 3.2 GUI Design

When designing the graphical user-interface (GUI) of this system, multiple prototypes are developed using the software proto.io. This gives an opportunity to analyse the potential flow of data and gain feedback on alternate layouts. This design stage is essential to fulfilling the project’s main motivation: developing an intuitive and easy to navigate system that is welcoming towards novices and quick to pick up.



(a) First Prototype with collapsible menu and full taskbar.

(b) Second Prototype with a divided interface and large lighting options.

Figure 3.2: Prototypes of the application’s interface when ran within a desktop browser.

### 3.2.1 First Prototype

The first prototype of the GUI shown in Figure 3.2a, has been designed with usability in mind by including these key features:

1. A collapsible menu allows the user to hide the model previews in order to have a full, unobstructed view of the created scene
2. Displaying the model previews within multiple pages, or a scroll-able menu, prevents the screen from becoming cluttered while allowing all features to remain easily accessible.
3. This design includes a placeholder to represent the preview icons for each available model. The user can drag this into the scene and choose the final positioning of the model.
4. The taskbar in the top centre of the window easily relays the available functions to the user. These buttons are identified with resemblance symbols to keep the system intuitive by relying on the user’s recognition rather than requiring them to recall the location of each option and their actions [23].

### 3.2.2 Second Prototype

The second prototype in Figure 3.2b, takes a slightly different approach:

1. A fixed menu splits the window into two, separating the area for world construction from the model selections. This removes any overlay, ensuring that the entire world is always in view.
2. By separating the taskbar and locating buttons within the menu pane, we prevent the screen from becoming too cluttered. However, this increases the reliance on the user to navigate through the menu pages.
3. The lighting buttons can be combined to present a single selection which cycles through the settings available - this follows the intuitive ordering given by the represented times of day.

- (a) The colouring and icon presented will also cycle to reflect the current lighting choice, to allow for quick recognition.
- (b) Any further lighting customisation developed in the future will be added to a separate page in the menu. This is an infrequently used feature so this extra navigation step will not disrupt the user’s workflow.

The save and load features will be implemented in a future development phase. Placing these buttons within the menu presents the concern that we will need a simple way to differentiate the “load a pre-saved scene” from “load in a model”.

### 3.2.3 Final Design

The final design is shown in Figure 4.27. It has been decided to follow a combination of these two prototypes. The menu has been created within an overlay across the world to supply a seamless and minimal appearance, rather than using disjointed sections.

This menu includes images of each available model, these are clickable buttons which place the model into the world so it can be dragged to its new position.



Figure 3.3: Final interface viewed on a desktop

In order to run on mobile devices the screen size must be taken into consideration. This requires a separate GUI design to ensure that the new restrictions do not hinder usability. For example, the icons must be large enough to be easily clickable but must not crowd the screen, and the menu must be hidden to optimise the space used for creation of the world [6]. However, we must still prioritise the frequently used features with quick accesses to not obstruct the users workflow.

## 3.3 Architecture Design

To optimise development, code re-use has been strongly prioritised in line with our Lean management approach, this aids in the reduction of wasteful processes and resource use.

To further the ease of maintenance, code modularisation is implemented through subprograms constructed with JavaScript’s ‘modules’ - each containing a library of functions needed for a core feature. This hierarchical breakdown decomposes the system’s complexity and – along with inline documentation – helps improve overall understanding of the code [24].

This process lends itself towards the objective of creating encapsulating functions to present the user with an abstracted approach towards programming with Three.js. By minimising interactions between the modules, and keeping them self-contained, we encourage the opportunity to use a decomposition of the project. This allows the user to restrict resource load by only including the features

to meet their current needs. It may be the case that they would rather use a subset of modules. For example, the opportunity to encapsulate the creation of an empty scene, removes the need for technical graphical knowledge, however the user may desire more control and customization on aspects such as the lighting.

Those who are new to programming in JavaScript may wish to re-use the logical functions, such as the character movements and interactions, within their own separate projects.

While main features are self contained, there is considerable interaction between some of the modules. As global variables can have significant impacts of the module’s reusability and code understandability due to name discrepancies [24], they have been avoided where possible. Instead of this, imported functions are used to explicitly pass data between subprograms, using getter and setter functions, however this introduces dependencies which may impact reusability.

### 3.3.1 Class Relationships

Some aspects of the system have been implemented using an object-oriented approach, these encapsulate significant aspects of the project, such as character attributes, path structures, and areas of repeated logic.

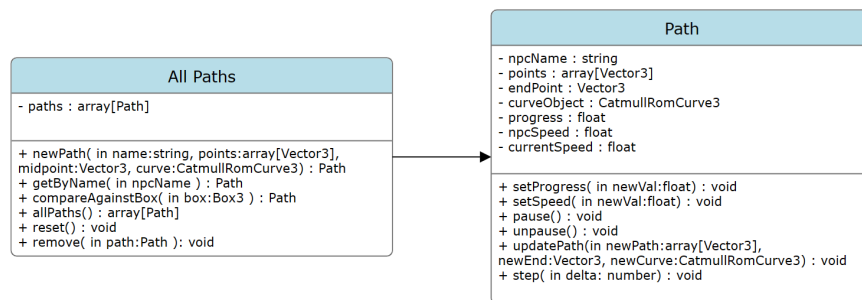


Figure 3.4: UML Diagram visualising the relationship between two classes. ‘AllPaths’ is a container for ‘Path’ instances.

These classes tend to be developed in pairs, for example there is an association relationship between the classes **Path** and **AllPaths**, as shown in the class diagram in Figure 3.4. The target class **Path** contains the main structure of a character’s path, including attributes such as the array of points the path is composed from, and the character the path belongs to. Whereas the source class **AllPaths** holds a collection of these **Path** instances.

Although it does not follow the single responsibility principle of modules, this design decision successfully achieves an efficient organisation of the logic surrounding **Path**. This creates a container class that provides interaction with all **Path** objects as a group and individually: all instances can be fetched or deleted, and the stored array can be navigated to locate the path corresponding to a given character.

A limitation of this approach is caused from the high dependency between these classes. Changing the constructor method of **Path** requires changes to also be made in **AllPaths**. This reduces the flexibility of the system, making it will be slower to adapt to changing needs as further functionality is added. Reusability is also limited since the classes are designed to work closely together [25], therefore it must be ensured that the code is properly documented to reduce future issues of maintenance.

On reflection, a more suitable design approach for path creation would be through the implementation a factory pattern. This would create a solution with higher maintainability as provides a re-usable encapsulation of the process [25].

## 3.4 Technologies

Three.js has been chosen as the main basis of this project due it being lightweight and capable of being hosted through a web browser. This ensures that the system is accessible to all, with no hardware requirements or prior installation. This is appropriate for users who wish to only use the fully graphical interface.

This API is developed with animation in mind, with in built methods to monitor the performance of the system when responding to extreme scaling of the number of assets rendered. The statistic graph `stats.js` allows for the visualisation of metrics such as rendering time or frames per second [26].

Three.js does not ensure backwards compatibility between the available versions, therefore the most recent version at time of development has been chosen: r157. This ensures that users are likely to have compatible systems, especially if they are installing Three.js for the first time. This assumption is made on the basis of our target audience being built from those with minimal graphical experience. Users with previous versions of Three.js will benefit from the support given by the Three.js online forums that aid those looking to mitigate to the newest version.

The main technologies and tools used to support the development process are as follows:

- Three.js addons:
  - `GLTFLoader.js` - Used to load in glTF models, including those exported from Blender and taken from the public domain.
  - `RGBELoader.js` - Used to load HDRI textures for environment mapping.
  - `OrbitControls.js` - Introduces navigation around the scene.
  - `Stats.module.js` - Adds an overlay panel to monitor code performance.
- JavaScript - Allows the creation of event-driven applications hosted within the browser.
- HTML and CSS - Used to customise the graphical interface and create interactive buttons.
- Vite - Locally hosts the application making it accessible through a web browser.
- Playwright - Used for automated testing and verification of the user interface and interactivity features.
- Github - Used for version control and regular backups at each development iteration.
- Blender 4.1.0.0 - This specialised software has been used to create the scenery and world objects, that the user positions to build up their scene.

## 3.5 Testing Strategy

A combination of manual and automated tests are put in place to validate that the core requirements are met [27].

### 3.5.1 Functional Testing

Throughout the development cycle, functional testing is continuously performed to ensure the verification of each component before they are integrated with the remaining parts of the system [28]. Unit tests are performed at the end of each module's implementation, to evaluate its functionality against the core objective that has been pulled from the work queue. This continuous testing reduces the risk of large issues appearing later on in development and helps improve the overall design and maintenance of the system [27].

Due to the large focus this project has on user interactivity, components are validated through the use of an API Playwright.

Playwright allows for the generation of tests based on a recorded interaction with the web page [29]. This allows us to simulate a user’s actions and validate that the correct changes have been made in response. The main application for this is to verify the load and display of the page elements, and to ensure that menu and taskbar buttons call the correct functions to elicit an appropriate response. For example, the character images must load the correct asset from the local server. This verification of correctness is essential to ensure that the user has a smooth workflow when using the application. For this, they must be able to confidently predict how the browser will respond to their actions [6].

At the end of each iteration phase, the developed module is integrated into the wider system and the group of affected modules are tested together. This incremental approach of integration testing identifies issues that may be raised by the interaction of these modules [30].

Once all modules have been developed and evaluated, the system is tested as a whole.

### 3.5.2 Non-Functional Testing

Part of testing the system includes the evaluation of its performance and responsiveness under different workloads [27]. This scalability testing explores how the system responds to a rise of data processing due to an increase in characters added to the scene [30].

This increases the computation needed by the GPU to render additional models, and must handle repeated process calls to calculate movement of the dynamic characters. Each character added to the scene must have a path generated, which may involve re-computation if new static objects are introduced. Each animation frame, the next position for each character must be computed, along with their animation clip, and any object avoidance that may be necessary.

This testing process takes advantage of the performance monitoring tools available with Three.js, in order to identify bottlenecks in the system [30], the `stats.js` helper library supplies a real time graph representing the frames per second on rendering the scene [26]. Google Chrome’s Development Tool allows for the measurement of key performance indicators, such as the render speed of each frame, CPU usage, and memory requirements [31].

Once this evaluation is complete, stress testing allows for the identification of the maximum limit of the system before complete failure [32]. Here, assets are continuously added to the scene until there is a significant decrease in performance, with aims of finding the maximum number of test objects that can be added to the world, and at what point the user’s experience is effected.

### 3.5.3 Use Cases

The final testing is be done manually and ensures the completion of various use-cases, during this browser performance will be monitored through Google Chrome’s development tool.

First we test the following typical use cases:

- U1 The user can launch the application within the browser of their choice by navigating to the local host. This loads the environment containing an empty world block with the “day” lighting setting.
- U2 The user can select from the scenery icons within the menu and add 5 objects into the world.
- U3 The user can add a character into an empty world. The user then starts the movement and allows the character to complete its path, until it revisits the starting position. The user can then pause movement.

- U4 The user can add an object into the scene and add a character. The user starts the movement and lets the path be completed as before, then pauses movement. The character should avoid this object.
- U5 The user can add character and play movement. They can then place an object onto its path, and the character should recompute its route to avoid this collision. Once the character has revisited its starting position, pause movement.
- U6 The user can add 5 characters into the scene, and then play movement. The characters should all travel along their individual paths, interacting with each other to avoid any collisions that may take place.

We also test extreme use cases to find the limitations of the system:

- U7 Incrementally add static models to a single world block until there is a significant decline in performance.
- U8 Expand the world significantly and add the maximum number of objects.
- U9 Incrementally add character models to a single world block until there is a significant decline in performance.

## Chapter 4

# Implementation

This section provides an overview of the development that has taken place to implement the key features that construct our Minimal Viable Product, as well as the added extension interactivity features.

### 4.1 World Creation

#### 4.1.1 Loading the Models

To begin the development phase, the first feature is pulled from the work log. This is the ability to load models into the scene [C1].

The development of this feature begins with an extensive research phase, exploring various modelling methods available, and the different file types compatible with Three.js. A large importance is placed on finding a fully supported format that gives the user the best real-time experience. Since the created Three.js scene runs within a browser, and hosted locally, it is vital that the chosen format is able to efficiently transfer assets despite the performance available on the client's machine. This ensures a smooth user experience even when using older computer models or mobile devices, that may have limited computational resources [33].

It was chosen to use glTF (GL Transmission Format) and glb file exports, since they provide compact transmission of an object and its texture. This gives faster load time compared to alternative OBJ exports which require separate MAT files to be included for the model's material resources [33].

To avoid security risks associated, the assets must be loaded from the hosting local server rather than giving the browser access to the client's hard drive. Because of this, the chosen models are hard coded into the program as a list of constants. This is outside of scope of the project, but limits the use of the system. A future development phase would allow the user to add their own models through a graphical interface navigating their directories. This would widen the project outside of the current use-case.

This project focuses on the use of low-poly models since this allows for the creation of recognisable objects without the sacrifice of resources. A lower complexity model requires much shorter rendering time, due to the reduction in polygons, vertices, and texture elements [34]. This is a significant performance risk for an animated project; when the characters explore the world, their models are re-rendered into the scene every frame. At any point of the time, the user is able to introduce new models, requiring the system to be responsive to the scalability of the scene. It is therefore essential that there is minimal buffer time in order to provide a seamless experience. Low-poly models aid in this since they reduce the scale of the data processed by the GPU.

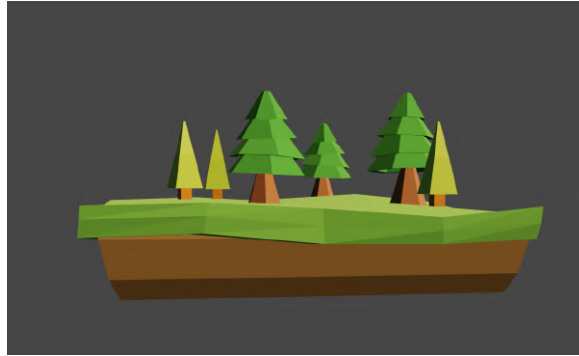


Figure 4.1: An example of a world the user can create, rendered in Blender.

### Blender Models

As part of the market research stage, Blender was chosen to be used to create the scenery objects that will build up the world. The produced models are shown in Figures 4.1 and 4.3.

Creating the models presented a first-hand experience using the software, giving the opportunity to isolate the key shortcomings that makes Blender so “daunting” for first time users, while also highlighting the areas of the design that make it so successful for those who are experienced [1] .

Version 4.1.0.0 of Blender was used, since this is the most recent version available at time of development, however version 2.8 and higher is necessary to develop models to use within the project, as this supports the exporting of glTF file types.

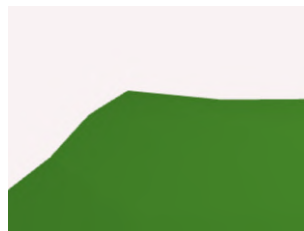
These models are added to the Three.js scene through the use of a [GLTFloader](#) [35]. Anti-aliasing is enabled within the [WebGLRenderer](#), to remove the staircasing artifacts that are introduced on the edges of the model [36]. The difference between the model render smoothness before and after anti-aliasing is shown in Figure 4.2.



(a) Model of world base as rendered with staircasing artifacts.



(b) A zoom in of the artifacts.



(c) Model post anti-aliasing.

Figure 4.2: A zoom in on the model showing the staircasing artifacts before and after anti-aliasing is added to the renderer.



All models are built alongside each other to ensure the scale and materials match, however each object is exported separately so the elements can be added into the scene at the users will. It is ensured that the origin of the model is localised before exporting, to ensure that there are no embedded translations that will interfere with positioning.

Further discussion on the placement of models is included in Section 4.4.



Figure 4.3: Premade scenery models available to the user.

### Character Models

The 3D models used for the interactive characters are sourced from the public domain, and taken from [quaternius.com](https://quaternius.com). These are provided as multiple file types, including glTF, and 12 animation clips are included with each model [18][19]. For this project, three of these clips are used to animate the characters movement, idle motion, and an interaction clip.

Animation of the characters are discussed in Section 4.2.6.

The models currently available in the system are shown in Figure 4.4.



Figure 4.4: Premade character models available to the user. These are of two types: human NPCs and animal NPCs.

#### 4.1.2 Exploring the World

The feature pulled into focus for the following development iteration is the user's ability to freely explore the scene [C6]. To do this, a camera is centred onto the origin which can pan across the world through the use of the arrow keys.

A second camera is positioned to look down onto the world. This provides a top-down view which – along with a grid overlay from the Three.js `GridHelper` - aids in placement of models that is not distorted from perspective. To swap to this view, the user toggles a Boolean flag through the selection of a taskbar button, or keyboard shortcut, which prompts the scene to be rendered from

the viewpoint and orientation of the other camera. This has been implemented through the use of an anonymous function called within an event handler, which references the current browsers window [37]. This achieves the extended objective [X3].

The two views available are shown in Figure 4.5.

In order to completely fulfil this main objective, the user is offered unrestricted zoom and rotation around the scene, through the use of the addon **OrbitControls**. This allows the user to manipulate the position of the main camera, viewing the scene from different perspectives. An ‘auto-rotate’ feature automatically orbits the scene, allowing the user to see their creation from all directions [38].

The combination of these features allows the user is able to freely navigate the scene without need of a mouse. Since the software is lightweight and ready to use within a browser, the application is able to run on all desktops and laptop devices. It is not yet feasible to run on mobile devices due to the screen size, this would require a separate GUI design to ensure that the icons are large enough to be clickable but do not crowd the screen, and that the menu is hidden to optimise the space used for creation of the scene [6]. However, frequently used features must have quick access to not obstruct the users workflow.

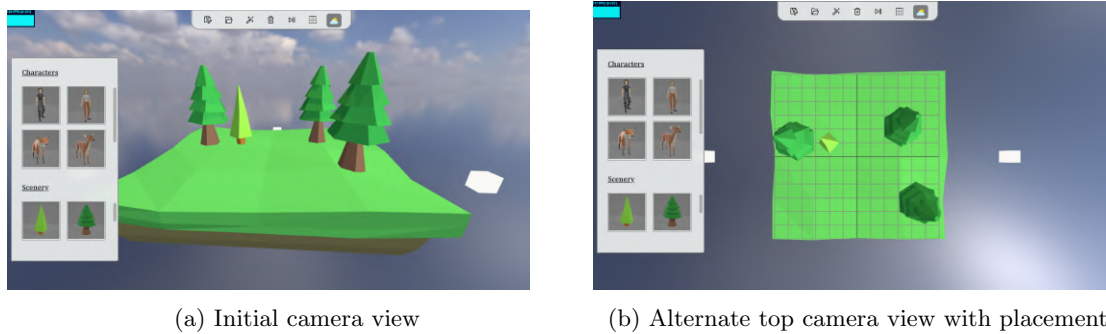


Figure 4.5: Camera views available to switch between, toggled by a hot key 'T' and an option in the taskbar

### 4.1.3 Lighting the Scene

In order to view the changes added to the scene, an ambient light source of low intensity is added to illuminate the world.

Image-based lighting (IBL) is used to provide a global illumination of the scene, while also providing a visual background. The environment maps chosen for this project are taken from [polyhaven.com](http://polyhaven.com) and [hdri-haven.com](http://hdri-haven.com), these are equirectangular high dynamic range images taken of real-world scenes. These texture images are projected onto a surrounding sphere to create a seamless skybox around the scene [39].

This adds realistic environment lighting to reflect different times of day and creates the sense of an infinite sky that the user's world can be expanded into. A lower resolution image is used to reduce the load time when switching between lighting settings.

When implemented within Three.js, IBL is not shadow casting. This is due to the use of a real-time rasterising renderer. This renders objects from the light sources' viewpoint, through the use of shadow maps: objects in view of the light are illuminated, while the rest are in shadow [40]. This technique cannot be used directly with IBL since the environment light can be described as a spherical function. This defines the lighting information for each incoming direction by distributing sample points across the surface of a unit sphere [41]. This means that there is not a directional source to follow to map out the shadows.

Therefore, in order to cast shadows throughout the scene an additional point light is positioned to act as our main light source, this immolates the sun.

This is where Three.js differs to dedicated modelling software, such as Blender, which often uses a ray tracing technique within its render engine [42]. This approximates the global illumination by computing multiple specular intersections from a ray cast from the camera's viewpoint. This allows for superior visual realism for shading and shadow production, as it determines how each object interacts with the light and deducts whether it is in shadow [43].

This technique is too computationally expensive to be used in real-time environments like Three.js, due to the large impact it will bring on performance.

The user has a selection of premade lighting settings that are available to cycle through, these dictate which lighting environment is loaded into the scene. Each HDR image is paired with a point light of differing placement, colour and intensity, to depict different times of day. The options available are displayed in Figure 4.8. These are provided within the taskbar, where the user can cycle through the different settings following the natural ordering. Icons given in Figure 4.7 have been sourced from <https://www.flaticon.com>. This fulfils both the essential objective [C4] and the extended objective [X2].



Figure 4.6: Using Three.js PointLightHelper to visualise the placement of the point light for each setting. Day is placed at the centre, Dawn at the top right corner, Dusk at the bottom left.

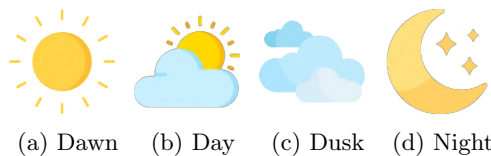
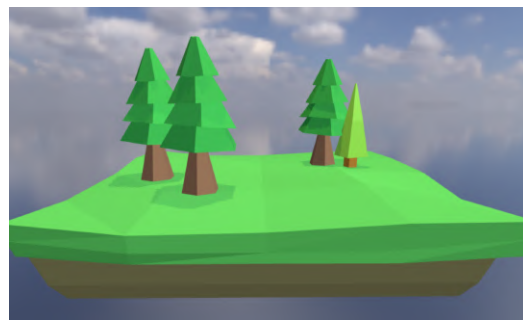


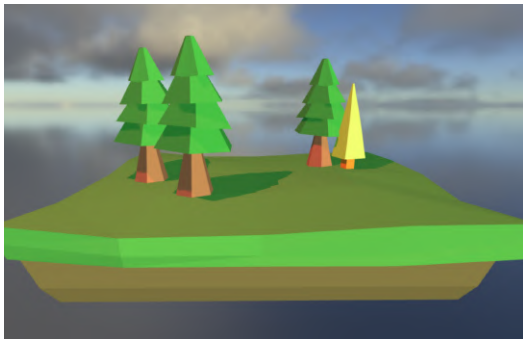
Figure 4.7: Icons representing the available lighting settings.



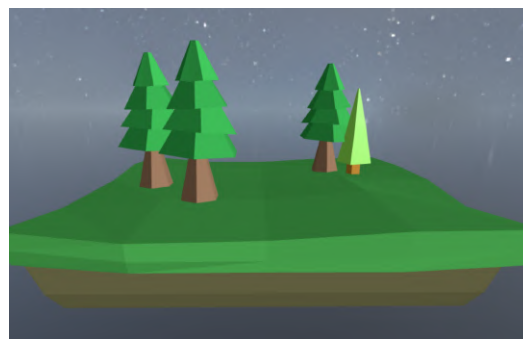
(a) Dawn  
The scene is lit from the right side of the world this casts shadows towards from the camera.



(b) Day  
The scene is lit from directly above, casting shadows directly downwards.



(c) Dusk  
The scene is lighted from the bottom left so the shadows are cast away from the camera.



(d) Night  
There is no directional light source, the scene is illuminated purely from ambient lights.

Figure 4.8: The different HDRs available and how this alters the appearance of the scene

## 4.2 Character Movement

The characters placed into the scene are free to explore the world by following random paths. These paths begin at the location chosen by the user, and travel to a randomly generated end point. In order to create natural and a seamless user experience, a collision avoidance system is used to manipulate the character's movement around other models placed in the scene.

This development phase focuses on the essential objective [C3] and its sub-objectives.

### 4.2.1 Static Collision Avoidance

Static objects are defined as scenery models the user has positioned into the scene, this also includes any large terrain changes (such as mountains or lakes). These collisions are handled during the generation of the path as have a large influence of the route taken.

This was originally implemented through the use of a spline to represent the characters path, whose anchor points are dynamically adjusted to avoid the intersections with the scenery models. This has limitations in accuracy; which prompts the implementation of a pathfinding algorithm.

### 4.2.2 Path Creation: Dynamic Splines

Initially, the random path is built based on a pseudo-random number generator to construct an array of co-ordinates the character should move to. The resulting curve is then split into multiple points which are analysed to ensure the path does not cross through a static object. An example spline path is shown in Figure 4.9.

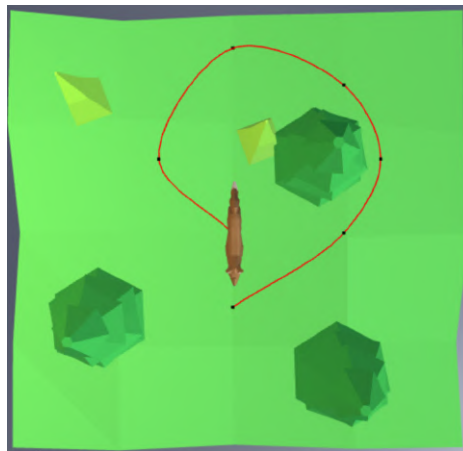


Figure 4.9: A character's path mapped out using a spline, constructed from the highlighted anchor points.

An axis-aligned bounding box is constructed from the mesh of each static object, using the `Box3` class supplied by Three.js.

Three.js also provides a `Box3Helper` which gives a visualisation of the bounding box, allowing a simple way to verify if collisions have been detected correctly. An example world with these boxes are shown in Figure 4.10. When the system is in verbose mode, these boxes are used to highlight the selected object, indicating which object is currently being modified. This reduces rate of error for the user, that may be caused by having the incorrect object selected.

The `Box3` class supplies various methods that aid in the detection of collisions. The main method used to implement this feature is `containsPoint`, this returns a Boolean value to signify if a given

point lies within the bounding box [44]. This comparison is done for each key point from the curve's geometry: if it is found to intersect the static object then the path is adjusted. The point in question is shifted along the appropriate axis, to ensure that the topology of the path remains the same. For example, if the point is found to be to the left of the box's centre, then it is shifted along the negative x-axis. An visual example of this process is given in Figure 4.11.

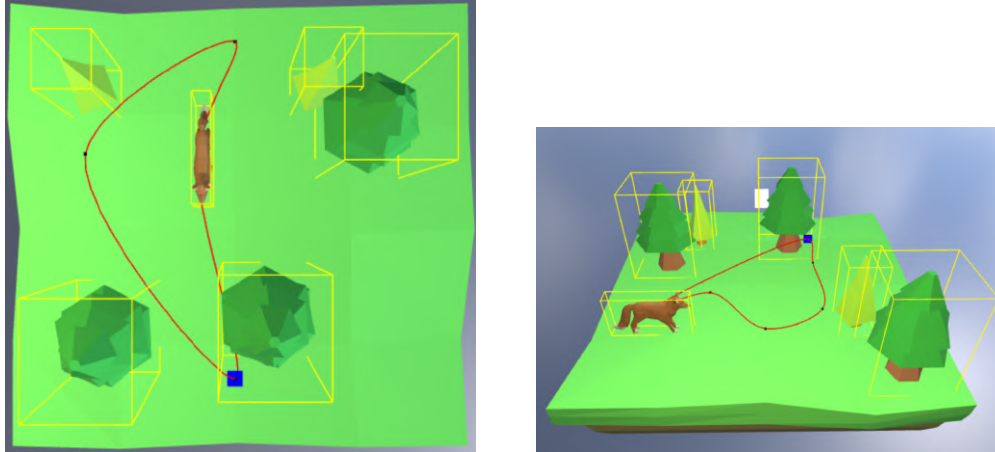


Figure 4.10: The highlighted point on the character's path has been detected to lie within an object's bounding box. We can predict that here the character will have a collision with an object. This is shown from both camera views.

These adjustments could be made in real time as the character progresses, however this would introduce redundant computation since the character is guaranteed to revisit this point and repeat the collision. Instead, the spline representing the characters path is broken down into key points based on its geometry, and each point is adjusted on path creation. This computation is done once per world configuration and is repeated only when a static object changes position: such as being translated by the user, or a new object is added into the scene. On this case, the points are revised once again, and are adjusted if necessary, to accommodate the new obstruction.

This process contains significant computation as each point is analysed in every path. This results in a nested count controlled loop, as shown in lines 4 and 5 of the pseudo-code in Listing 4.1. Computation increases exponentially as more dynamic models are added to the scene.

Listing 4.1: Revising Paths with Dynamic Splines

```

1 # Triggered on a 'mouseUp' event after dragging an object
2 If selected object is a static object:
3     Get bounding box of the object
4     For each path in AllPaths:
5         For each point in path:
6             If box.containsPoint(point): # Method from Three.js box3
7                 Adjust point to step out of the box
8                 Reconstruct path using the new point
9             End If
10        End For
11    End For
12 End If

```

When evaluating this approach, it is determined that the accuracy is dependent on the number of points adjusted. Consider the case where only a few points are included, this will not guarantee the path to have zero collisions. If the anchor points are found to be on either side of the object, then the curve connecting them may pass through the object's bounding box undetected.

This introduces a trade off between accuracy and performance. Introducing more anchor points allows for a thorough analysis of the path, but increases the already expensive computation required. This is shown by comparing the results in Figure 4.12a which revises a path using 7 points, to the path produced in Figure 4.12b using 50 points.

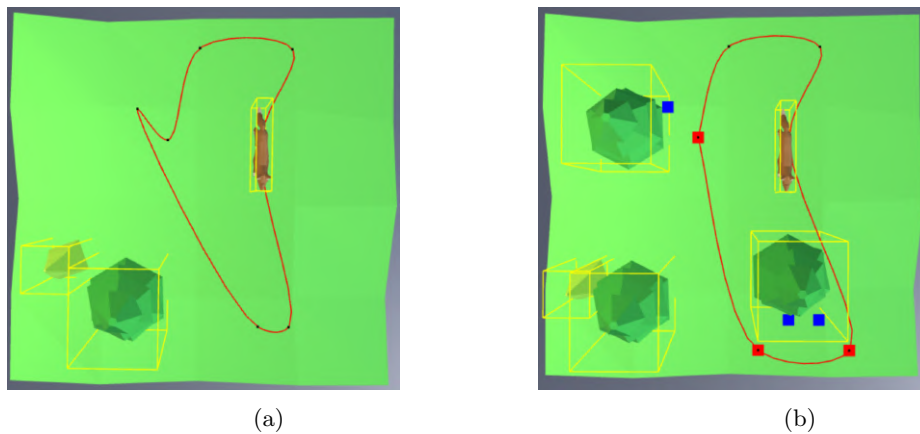


Figure 4.11: An example use case adjusting the character's path in (a) by placing additional trees into the scene (b). The points that intersect with an object's bounding box are indicated in blue. These have been re-positioned to the points in red, and the spline is reconstructed using these.

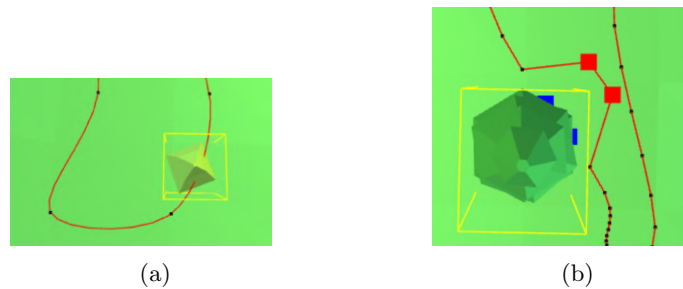


Figure 4.12: (a) A small number of points on the path results in many collisions being missed. (b) A higher number of points produces a collision avoidance system with higher accuracy.

This approach limits the overall performance of the system. For example, consider the world given in Figure 4.13. When a character is placed into a world with existing objects, a path is constructed from randomly generated points. Then each point on the curve is iteratively compared against all existing objects, through the use of an additional similar nested loop.

In the following section, we discuss a computationally superior method which combines these two stages, and produces higher accuracy.



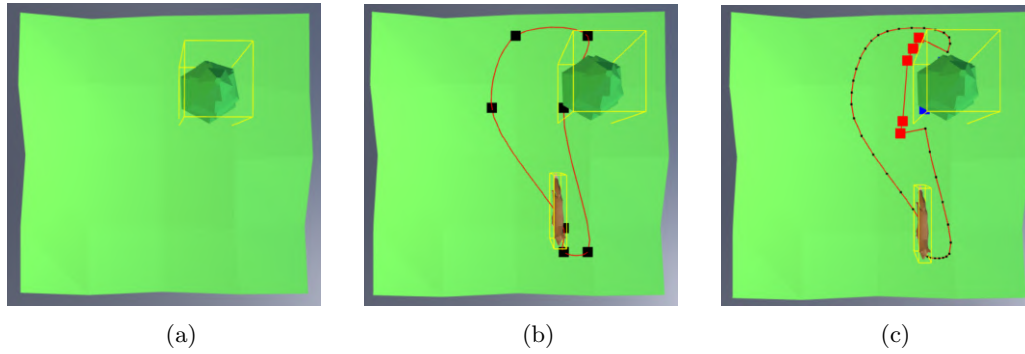


Figure 4.13: Stages of computation when adding an object followed by a character. First a path is computed from randomly generated points (b), this is then segmented and compared against static objects (c).

### 4.2.3 Path Creation: A\* Algorithm

Subsequently, a differing approach is explored relying on an open source implementation of the A\* algorithm [45].

For this, a graphical representation of the world is kept. Each node represents a possible position that the character can move to, weighted by the current occupation state. When computing the path to follow, only nodes that are not currently occupied by an object will be considered - this is a similar approach to that followed by Unity from a Navigation Mesh of walkable surfaces [11].

To ensure that a character does not clip through an object's mesh, the world graph must reflect the entirety of the positions that have been removed. The bounding box of the object is used to retrieve the size of the object and thus the number of nodes that must be updated.

An example of a world and its representative graph are shown within Figure 4.14.

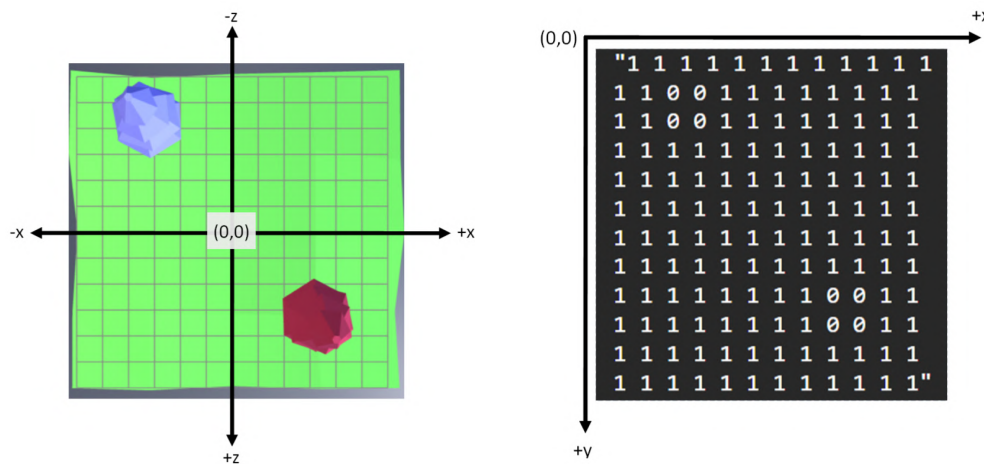


Figure 4.14: The graph representing the objects currently placed in the world. This is visualised as a 2D array, where each index maps onto a co-ordinate position within the world boundary. A value of 1 represents an empty node, a value of 0 indicates that this node is occupied therefore a character cannot be moved there.

The weighting of appropriate nodes are updated through the access of a 2D array, indexed by a projection from the object's world position onto the 2D co-ordinate system. The **Map Range** formula [46] normalises the 3D vector within the range (0, width of the world).



$$t = b_1 + \frac{(s - a_1)(b_2 - b_1)}{(a_2 - a_1)} \quad (4.1)$$

This maps value  $s$  in range  $[a_1, a_2]$  onto a value  $t$  within the range  $[b_1, b_2]$ .

Since both ranges have the same width, this equation can be simplified.

$$x_{new} = z_{old} - min_z \quad y_{new} = x_{old} - min_x \quad (4.2)$$

Where  $min_z$  is the minimum z-value of the world block the object is currently positioned onto. For the block placed at the origin, the value is mapped onto the range  $[-\frac{width}{2}, \frac{width}{2}]$ . Finishing the projection, the two vector values must be swapped to remedy the inversion of the z-axis, caused by traditional camera positioning.

At any point in time the user may introduce a new object into the scene, or reposition an existing one. In response, the world graph is updated to reflect the current positioning of objects and the paths of nearby characters are recomputed. This ensures no new collisions being introduced.

By forming our paths from the original start and end positions, the resulting path is very similar to that of the character's original route. This is due to the heuristics of the A\* algorithm, and solidifies the natural movement and response of the character's.

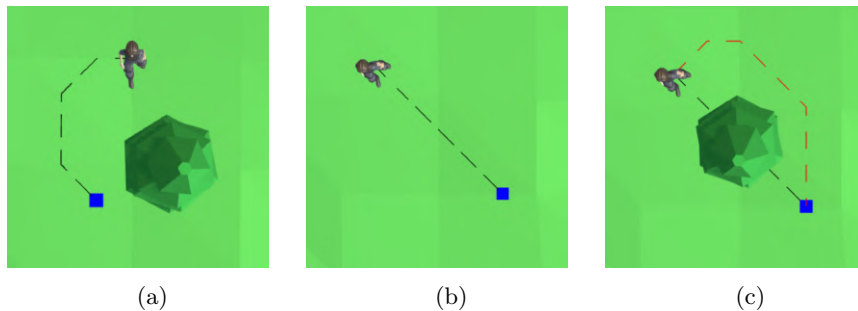


Figure 4.15: (a) The creation of a character's path, taking into consideration the position of pre-existing objects. (b) A randomly generated path for a character placed into an empty world. (c) An object is then placed onto this path and the route is diverted around it.

Following this method, there is no cause for additional computation after the generation of the character's path due to the nature of the path-finding algorithm. This has higher efficiency than the previous approach, which separates the stages of path creation and collision avoidance.

Currently a limitation is placed onto character movement by constraining their paths within the chosen world block. This reduces potential waste of resources caused by a single dynamically sized graph to reflect world expansions. For extreme world shapes, this technique would produce areas of the graph that the characters have no way of accessing. For example, the world given in the Figure 4.16, would produce a large 2D array with two blocks of unused space as shown.

Instead, a separate world graph is kept for each world block added, shown in Figure 4.17. In the future this limitation could be reduced by exploring the impacts of this method and whether it is avoided by adding world size constraints.

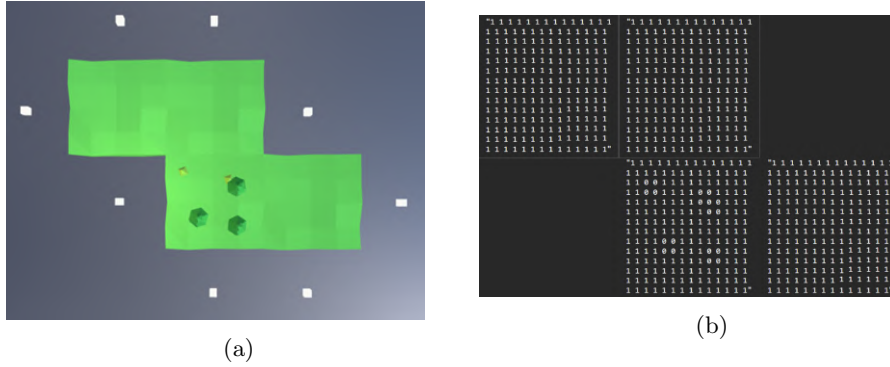


Figure 4.16: An example expanded world and the 2D array representing its position graph.

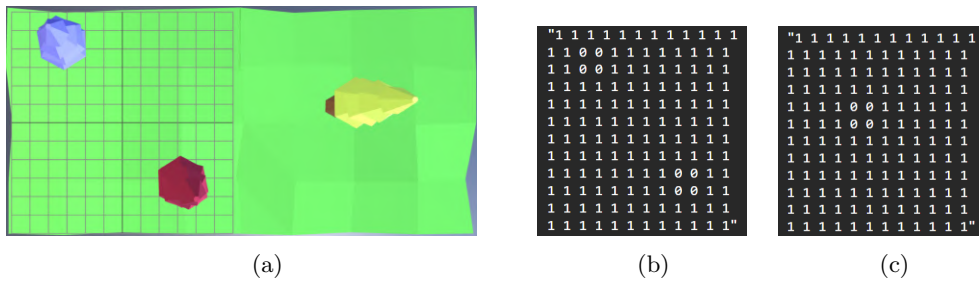


Figure 4.17: An example of an expanded world (a). Two graphs are used to represent the occupation states of the two world blocks, these are visualised as 2D arrays. Figure (b) shows the object positions in the original block, figure (c) shows the additional block.

#### 4.2.4 Generation of a Random Goal

It could be said that following a completely random path creates unnatural movement. One of the reasons for this is due to the paths being too short.

To solve this issue, the end point generated must be restricted within the ranges shown in Figure 4.18. This ensures that the character's path is longer than a minimum value, but does not go past a maximum - for now this is bounded by the size of the world block.

For example, the x-value of an end point of a path with a starting position of  $x_1$  must be bounded within  $0 \leq x_1 \leq (x - min)$  or  $(x + min) \leq x_1 \leq worldwidth$ . Where  $min$  is the shortest length a path can take.

The first approach considered is through a pseudo-random system which prioritises movement in a particular direction. In the case that this moves the character outside of the world boundary, the path is generated along the inverse of the chosen axis. While successful, this causes an unnatural repetition with the movement of characters.

A more computationally expensive method helps to vary the routes these characters travel. This takes a direction into consideration only if a path of minimum length along this axis will not bring the character outside of the world. This deals with the edge cases given in Figure 4.19. Of these, a random X and Y direction are chosen, along with corresponding boundary values, and the paths end point is generated within these.

For example if the positive x-axis and positive z-axis are chosen, the end point is positioned at a random co-ordinate in the bottom right quadrant of the world. An example solution from this is given in Figure 4.20.

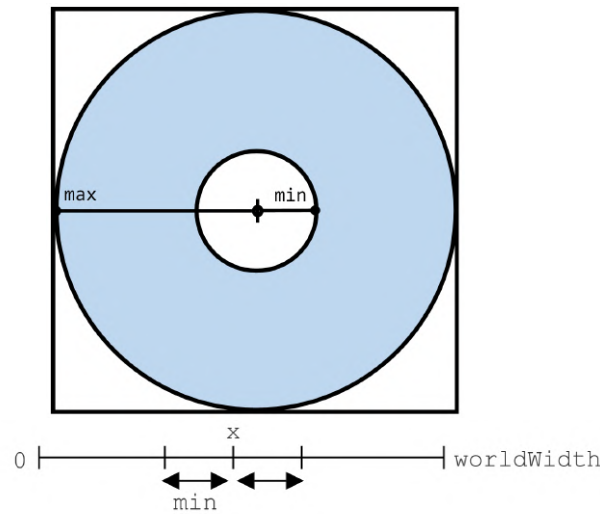


Figure 4.18: The highlighted region visualises the possible positions a character's path can end, to fulfill a minimum length.

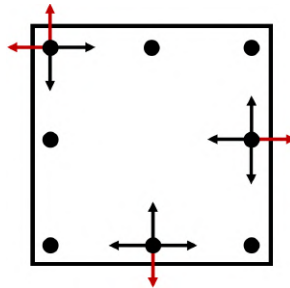


Figure 4.19: The 8 edge cases of the character's position when considering which direction they can travel.

A future development phase will explore a less computationally extensive approach. This generates a random angle to dictate the direction for the character to move along, and a random euclidean distance between the minimum and maximum ranges. This restriction ensures that the goal generated is with the annulus representing valid path lengths. The logic for this is shown within Listing 4.2 and Figure 4.21. This does not reliably handle edge cases therefore more research is necessary before implementation.

Listing 4.2: Calculate Random Goal

```

1 Function calculateGoal():
2   let x, y = starting position
3   # Generate a random angle to move along
4    $\theta$  = random value (0, 360)
5   # Generate a random euclidian distance
6   let d = random value (min, max)
7
8   # Calculate the change in x and y values required to reach this end point
9   let  $\Delta X$  =  $d \sin(\theta)$ 
10  let  $\Delta y$  =  $d \cos(\theta)$ 
11
12  # Calculate position of goal
13  let goal = 2DVector( x+ $\Delta x$  , y+ $\Delta y$  )
14  Return goal
15 End Function

```

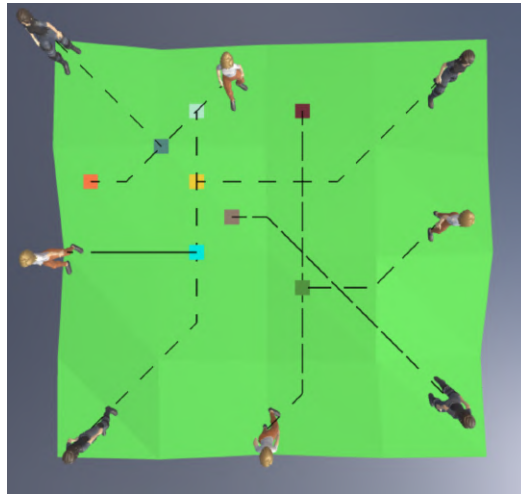


Figure 4.20: Random end points generated for each character placed at edge cases along the world boundary.

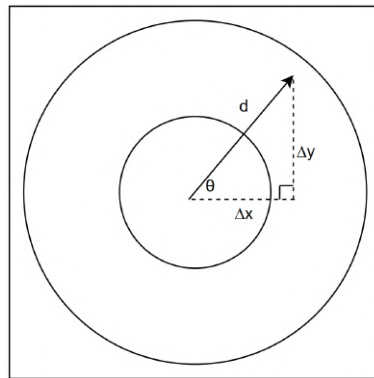


Figure 4.21: Approach to calculating a random path goal through generation of a random angle.

#### 4.2.5 Following the Path

Once the end point is generated, a route to this is constructed based on the character's type. An example of these paths are shown in Figure 4.22.

- Type 1 NPCs (humans): These characters have one part to their path, generated through the A\* algorithm. Progress along this is then reversed once the end point is reached, so that the start position is revisited.
- Type 2 NPCs (animals): Two parts of the path are constructed, this has the start and end points inverted, and are combined to produce a circular path.

In order to translate the character along its path, a spline is built from key weigh-points where the NPC changes direction. Every render loop, the next consecutive point is retrieved from the curve using the method `getPoint`, from the curve class. This takes in a floating-point value in the range  $[0,1]$  [26] - this is a representation of the current progress the character has made along the path so far. This progress variable steps through the points on the curve based on the clock delta and characters set speed.

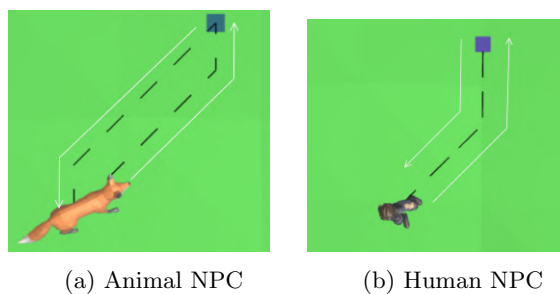


Figure 4.22: Example paths for each character type.

### 4.2.6 Character Animations

Three.js has a similar animation system as Unity [47], which has been discussed in Section 1.2.2. We must instantiate separate instances of **Animation Mixer** for each model, in order to allow simultaneous independent animations [48]. A character class is created to hold reference to this mixer and its relevant clips, this ensures that each character can access their corresponding mixer to update their current action [47].

The character has four potential phases, the transitions between these states are shown in Figure 4.23. There is a boolean flag, that the user can adjust, which controls all movement in the scene. When the world is first created this is set to False, and all added characters are in a ‘wait’ state. Once the user plays the movement, the characters then begin to explore the scene and enter their ‘walk’ phase. This triggers the corresponding animation, and sets their current speed to dictate how quickly they progress on the path.

At any point the user can pause all movement in the scene and the characters will return to their ‘wait’ state - here animations and path progression is paused. The diagram shows three loops onto an ‘Idle’ state, this is triggered by a user click onto a character. This stops movement and plays their Idle animation. After a second click, the character will return to its previous state.

If a character is clicked on while all movement is paused, then it will not move until the user clicks it again, even if the overall movement of the scene has been resumed. This allows the user to decide which characters will explore the scene, and which will stay static.

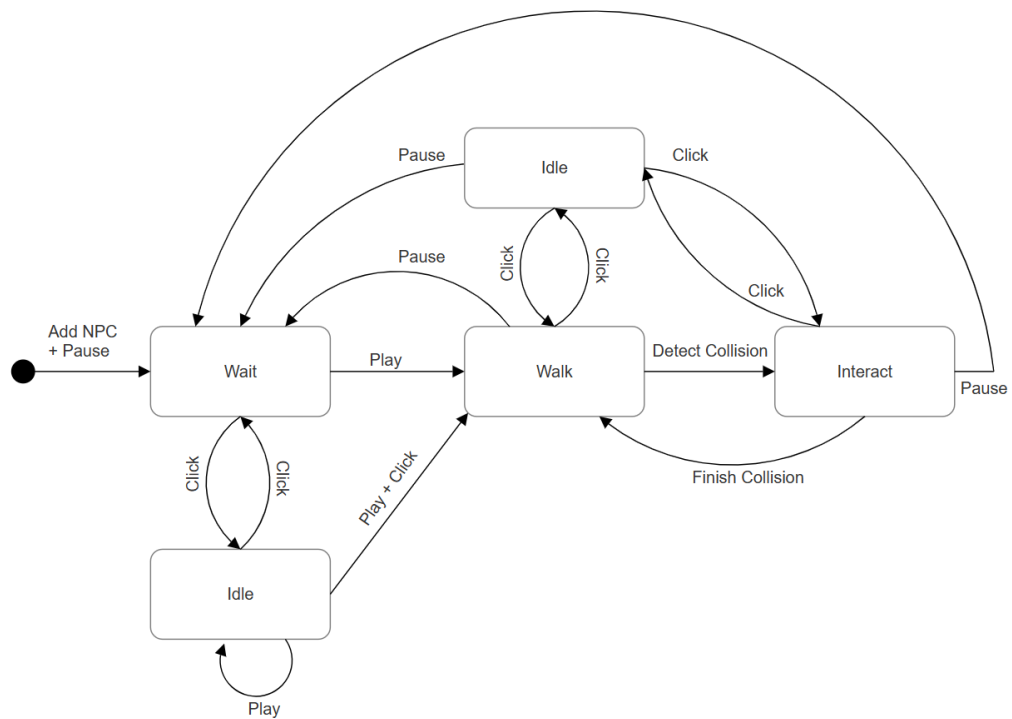


Figure 4.23: State diagram of the transitions between a characters phases.

## 4.3 Character Interactions

When characters explore the scene, it is necessary for them to avoid collisions amongst themselves as this causes the models to unnaturally clip through one other. This essential feature can be achieved through the use of dynamic collision avoidance, which adjusts the paths in real time.

### 4.3.1 Dynamic Collision Avoidance

Once a potential collision has been detected, the actions taken to avoid this will be applied based on the type of NPCs that the interaction is between:

#### Interactions of the Same Type

The characters pause progress along their path, rotate to face each other and play an animation between them. Once this has finished, the movement of each character will be resumed, using a staggered delay, to ensure that the collision will not recur.

#### Interactions of Differing Types

The character involved is moved to a point away from the collided character, and then will rejoin onto its original path through a new route. This is done following a similar implementation as discussed in Section 4.2.2: the position of the character is adjusted based on its relation to the centre of the other's bounding box. In this case, this was found to be less resource intensive than running the A\* algorithm as only a small path is needed.

This approach is limited as does not always achieve a completely smooth avoidance, occasionally the character meshes will clip through each-other upon playing their animations. This is since the size and speed of the other character is not taken into consideration, instead the system currently uses constant values for all interactions. The next development phase will focus on fixing this bug to ensure natural movement on the avoidance of collisions. Pseudocode of the current implementation can be found within the Appendix A.



Figure 4.24: A visual of the rays cast from the character, this is what they can “see”.

### 4.3.2 Approaches Considered

This requires a different approach to the static collision avoidance discussed in Section 4.2.1 since the avoidance system must be implemented during the movement of the characters, rather than on the generation of the path. This is since the position of the character's involved will not be known until they are within range of one-another.

The first approach increases the size of the model's bounding box to include the visible area. If the position of an NPC intersects this box, we can assume that the probability of a collision is high. This method follows a similar approach discussed in Section 4.2.2 and has similar drawbacks as we iteratively compare a character's position with all others in the scene.

An alternate approach is the use of raycasting [49]. A ray can be sent in the direction the character is facing, and the probability of a collision increases if this intersects with another model. If the length of this ray is set to the threshold range between characters, a collision is detected only when within visible range. Sending out three individual rays, each separated by an angle  $\theta$ , increases this visibility to include characters that are approaching from the side. This is visualised in Figure 4.24 using the Three.js `LineHelper`.

It was decided that this approach brings the most natural character interactions, and sacrifices the least complexity.

To prevent characters from becoming stuck within a “detected collision” loop, an attribute `cannotInteract` has been introduced. This flags when a character is involved in a dynamic collision, and is reset after 9 seconds have passed - this gives enough time for the interaction to be completed, and for the character to find subsequent collisions. This avoids immediate repeated interactions which prevents them from progressing.

A future development phase would explore the approach of a force-based avoidance mechanism, this could be implemented through the use of a physics engine such as `enable3D` which provides a gravity system. Once the characters become within range of each other, a force would be applied to repel them apart. This would avoid the notion of a deadlock between characters as repeat collisions are unlikely to occur due to large corrections.

Research from Reynolds has found that while force field models often produce successful results, there are unavoidable drawbacks. He highlights the case where an obstacle approaches a character at such an angle that is directly opposite the applied force. This causes the obstacle to slow on its path but will not turn to move away. Another instance is when an object may be within the force field, but does not have potential to collide, in this case the character should be free to travel alongside it [50]. He goes on to discuss his preference for “long-range planning” over “last minute” corrections [50], which is often unreliable.

This shows that there are benefits in exploring this method, however more research is needed to ensure its reliability.



## 4.4 User Interactivity

Before entering the development phase surrounding the user interactivity features, a survey is constructed for participants to rank typical features found in design software based on what they deem to be the most “useful”. This allows us to add an additional ordering to the interactivity features, to ensure that those with the biggest influence on the user experience are prioritised. A sample of the results from this survey is given in Figure 4.25, this has given highest priority to the free placement and removal of characters, over finer customisation such as editing colours.

The survey also asks how a user would expect to trigger these key features, including where they would expect them to be found – for example a hot key they would first attempt, or what menu page they would search in. This aids in keeping the interface as intuitive as possible, by organising the features based on the user responses.

The results of this survey are given in the Appendix B.

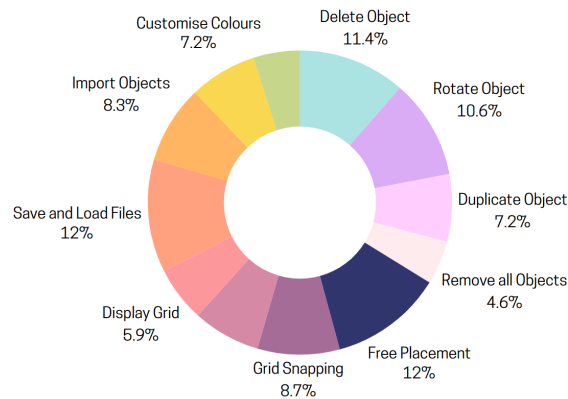


Figure 4.25: Sample of features that users have expressed a need for.

### 4.4.1 Adding Models to the World

The main objective of our project is to allow the user complete customisation on the position of premade models [C1] and [C2]. Originally, it was planned for the object menu to be implemented through a drag and drop system, this would allow the user to drag the object directly from the menu onto the chosen position in the world.

However, since the menu is constructed from HTML button elements, the introduction of models into the world is done through the windows event handler.

On a ‘click’ event of a model image, the object is loaded into the scene outside of the world boundary. The `mouse_move` event then begins to listen for the user’s movement so that the object can follow along with their cursor. Once the `mouse_up` listener receives a response, the object is placed down onto the selected position.

A depiction of the logic of the instances of event handlers is given in Listing 4.3.

During user acceptance testing, it was found that the majority of users will first attempt dragging directly from the menu, however they will quickly notice the error and correct themselves without prompting. While this adds a small delay to the users first experience of the application, the learning curve is negligible. The “drag” experience is emulated by regularly updating the position of the model so that it moves alongside the mouse cursor. This gives a clear visualisation so that the user can select a point on the world to drop this into the desired position.

### 4.4.2 Selecting Models

On a mouse click, an anonymous function is called which uses ray-casting to compare the cursor position against the objects that the user is able to interact with. This returns a reference to the intersected group of meshes, the hierarchy within the `Object3D` class can then be traversed to find the main group encapsulating the object. This is the reference used to translate the entire object without splitting meshes.

### 4.4.3 Translating Model Positions

After the selection of an object, an event listener is created to update the position of the object based on the location of the user's pointer – this is removed on the event of the mouse being released, which emulates the 'drag' action.

A two-dimensional surface is constructed parallel to the X-Z plane, at the point  $Y=2$ . This is configured by declaring a unit vector to define the surface normal as being along the y axis, and is shifted by a constant away from the origin [51]. An example of this plane is given in Figure 4.26.

In order to move the dragged object so that it is stuck along this plane, a ray is cast down from the mouse pointer to find the position that this intersects with the plane – this is the point that the object will then move to. This creates an illusion of the cursor directing the object to float above the worlds surface, rather than being haphazardly stuck to the cursor.

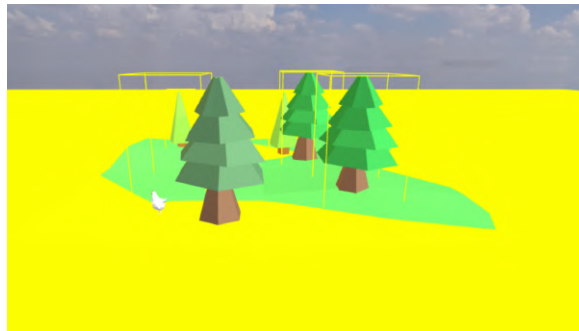


Figure 4.26: An example plane at  $Y=0$  that the character will move along. Movement is then restricted to only move above the world.

It must be ensured that the object is only placed within the boundary of the world.

When dragging the object, a ray is cast along the negative y-axis centred from the mouse pointer. If this ray is found to intersect with the world block, then it is known that the object must be floating above its surface. Therefore, the position of the object can be updated.

The object must be placed flush against the world block, despite the varying levels in terrain.

On confirmation of the object's placement, an additional ray is cast downwards to return a structure containing all objects that are intersected along this ray as well as the distance it has travelled to reach them [52]. The Y value of the object can then be decreased with this, bringing the object down to sit against the world's surface. This ensures that the object will not be floating above the world or embedded inside it.

Once the object has been placed, only its current position is stored and the graph representing the state of the world is updated. This ensures that the previous positions will not be taken into consideration on generation of a characters path, as these locations are no longer blocked.

Listing 4.3: Logic of Event Listeners

```

1 # On load of scene
2 Add Event Listener MouseDown
3
4 # On mouseDown event
5     If click on object then
6         Add Event Listener mouseMove
7         Add Event Listener mouseUp
8     EndIf
9
10 # On adding an object
11 Place character into the scene
12 Add Event Listener mouseMove
13 Add Event Listener mouseUp
14
15 # On mouseMove event
16 Update object position to the pointers position
17 Add Event Listener mouseUp
18
19 # On mouseUp event
20 Place object onto selected position
21 If selected is a static object then
22     Update world graph
23     Recompute paths of intersecting characters
24 EndIf
25 If selected object is a character then
26     Generate path
27 EndIf
28 Remove Event Listener mouseMove
29 Remove Event Listener mouseUp

```

#### 4.4.4 Clearing the World

While each object can be individually deleted through the use of a hot-key, a “bulldoze” shortcut allows the user to clear all objects from the scene through a button in the task bar, this can be seen in Figure 4.27.

This prompts a function to loop through the scenes hierarchy, to remove each child individually, this must first gain confirmation from the user through a JavaScript alert. Appearance could be improved by implementing this through Three.js sprites.

This essential objective [X5] accelerates the users work flow and allows them to easily restart world creation from a blank canvas.

#### 4.4.5 Generating Scenes

A separate button, seen in Figure 4.27, allows for the generation of a random environment, this removes all models currently loaded into the scene, and generates 10 random scenery objects and 2 random characters. These are then assigned pseudo-random positions, while ensuring that multiple models cannot be placed at the same co-ordinates. These scenes render within a second, and the user is able to repeat the generation as many time as they wish. Once the desired scene has been produced, the user can continue creation as if they had built the world from scratch. [C5] They are able to resume movement of the characters to see how they explore the scene and interact amongst one-another, and can add and remove objects as they see fit.

## 4.5 Graphical User Interface

The graphical interface shown in Figure 4.27 has been created using a HTML overlay, customised with CSS style sheets.

Alternatively a graphical menu can be produced using the Three.js built-in object **Sprites**.

These are 2D images that can be overlaid above a scene and will always face the camera. Often these are used to add particle effects, or used as a lightweight alternative to creating a 3D model [53]. Sprites can therefore also be used to create buttons that will appear with a fixed position over the scene, and are not impacted by movement, if rendered onto a separate orthographic camera. They can be made interactive through the use of raycasting, however this introduces significant complexity that could otherwise be avoided [54].

It was decided to use HTML paired with CSS to construct the graphical overlay as it has found to have higher flexibility and is more maintainable than the alternate Three.js approach. This is due to HTML being thoroughly documented and its features are less likely to become redundant due to the backwards compatibility that can be achieved by following the methods discussed in Section 5.5.1.

This follows the structure designed in Section 3.2 .

To meet the extended objective, a side panel menu has been included to clearly display the previews of all available models that the user can add to the scene. [X1] Currently, this can only be customised by directly modifying the model references in the code, however the next development phase will focus on adding an interaction for this within the GUI. This will streamline the users actions, and strengthen the achievement of the projects main motivation.

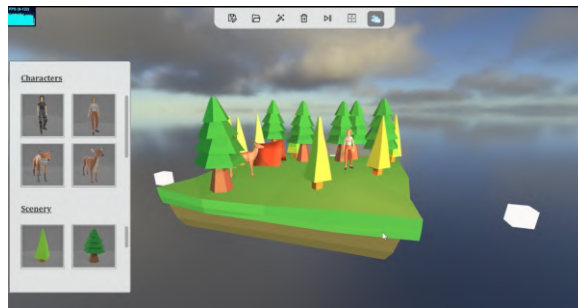


Figure 4.27: The graphical overlay constructed in HTML, providing buttons for each key feature.

Extended interactivity features have been included within the taskbar buttons and are triggered on a click event.

A minimalist button allows the user to cycle through the lighting settings, following an intuitive ordering: Day, Dusk, Night, Dawn. [X2] A CSS responsive event is used to update the button's attributes, such as updating the icon that is representative of the current setting selection. While the use of icons rather than textual menus helps to declutter the screen and relies on the user's recognition for navigation, in order to ensure usability we must follow the icon-specific guidelines given in Sheiderman's usability guide. This includes ensuring that icons are distinctive from each other and that there is a harmony between the icon family [6]. For example, icons have the same line weight and size to ensure that the page appears coherent and thought-out .

The taskbar gives the opportunity for the user to toggle camera views and character movement; and gives access to the extended features of bulldozing the world and generating randomly built scenes. Placeholder buttons have been included to indicate the locations of the save and load functionality, so that it can easily be integrated into the system once developed.

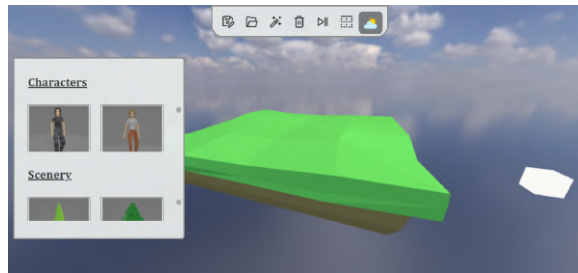


Figure 4.28: Final interface when viewed on a mobile device, iPhone12 Pro

Using Google Chrome’s Development Tools we can visualise the current design when ran on a mobile device, this is shown in Figure 4.28. While there is sufficient space to build up the world, the menu is unusable due to shrinkage and the taskbar buttons are too close together for accurate interactions. Once the application has finished its development, a mobile version will be considered to further increase the potential user-base. This will need modifications on the handling of interaction events, and extensive research on the performance of the system and the resources required to run it successfully. It is likely that a lightweight version of the application will be used for mobile devices, this will lack the resource-intensive features that have minimum impact on usability, such as shadow casting and pixel resolution.

A purely minimal version of the application would be used to visualise creations but restrict the customisation the use has on the world. A user would be able to explore their created scene and toggle the movement of their characters. This allows for the main applications of the project to remain but a desktop computer would be necessary to modify the world.

A mobile version of the software presents an opportunity for an online platform that allows users to share their creations: gaining positive feedback and support from those who have more experience with 3D modelling. This, along with a ‘Help’ feature would widen the scope of the project and strengthen the software’s use as a learning tool.

## Chapter 5

# Testing and Performance

In this section we discuss the final testing of the project verifying our non-functional requirements and the main motivation of the project: creating an easy to use browser based software that is more welcoming to newcomers than the professional alternatives available.

### 5.1 User Testing

At the end of the development of the project, a demo session allowed potential users to trial the product and give feedback on its current usability. This session was conducted with a group of ten students with varying experience of computer graphics and 3D modelling. They were asked to first explore the software independently, trialing each available feature, and then were welcome to construct a personalised scene or reproduce a given example.

On monitoring their behaviour with the software, we were able to see which features were the most commonly used, and look for any aspects that caused frustration.

#### 5.1.1 Usability of Controls

It was a shared consensus that use of a mouse greatly enhanced the user's experience compared to a laptop track-pad. This often impacted the ability to pan and explore the scene smoothly due to difficulties with holding a right-click. While the arrow keys are available to pan the camera, many users did not think to try this. It is thought that a welcome screen listing the available hot-keys will help the user to quickly become familiar with the functionalities available.

To contrast, users were found to quickly pick up the mouse controls and could successfully navigate the world. Many expressed that these provided natural movement and praised the similarity to the controls of other software available. As provided by `orbitControls`, the left key allows free rotation of the scene, and holding the right button allows the user to pan the camera around the scene. When using the top camera, the user can move between the placed world blocks through the use of the "WASD" keys.

It was suggested that the panning speed of the camera is too fast, making it difficult to maneuver to a specific point to gain the desired view of the world. This will be taken into consideration with high priority since accurate camera positioning is essential for potential applications of the system, such as use as a design tool for modelling perspectives of a world. A settings menu would allow for the modification of features, such as control dampening, allowing the user to adjust the responsiveness of the system. Since this would be very infrequently accessed, it is best to hide this menu in the top corner of the screen so that it does not cause unnecessary distraction when not in use.

For users with no graphical design experience, observation showed a large contrast in attitude towards our software compared with tasking them to use Blender or AutoDesk3D for the first time.

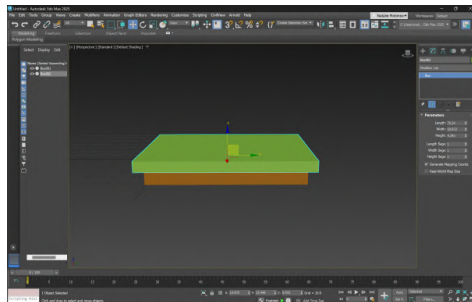
Users often preferred our navigation system compared to that of Blender, stating that the complexity of Blender’s controls did not come natural to them. The same consensus was given when using AutoDesk3D, this often had points of frustration for the user as they frequently had the incorrect tool selected when attempting simple tasks such as panning the camera or translating an object.

### 5.1.2 Usability of the Interface

Many users commented on the interface of the application and all feedback received was positive. When asked for initial opinions on first viewing each of the interfaces, as seen in Figure 5.1, users described the interface of AutoDesk3D through keywords such as “technical”, “cluttered”, “professional”. On the presentation of our project the users chose descriptors such as “simple”, “casual”, and “vibrant”. Users who had no experience in 3D modelling, agreed that the professional systems gave a sense of overwhelm when first approaching the system, due to the complexity of features available.

It was agreed that our project is more welcoming upon first impression, and appears as a casual idle game compared to more serious software. This is much more approachable for a wider audience.

To describe the creation process within both applications, the test users complimented our system as “easy-going” and natural to use. They explained that they prefer the simple approach towards 3D modelling, since they currently have no need for complex tools, and would like the opportunity to view later versions of this project once it has been developed past the current scope.



(a) AutoDesk 3D



(b) Our Project

Figure 5.1: Graphical interfaces tested by the users.

When prompted for potential areas of improvement, it was suggested that usability would be benefited by the choice to customise the GUI, and many users agreed. While currently not within scope of the project, options such as repositioning the menus and hiding them away while not in use, would allow the user to tailor the workspace to suit their own needs.

The current layout of the menu gained mixed responses. Some would prefer to have multiple pages separating each object type, rather than the current divided scroll containers. We agree that this would provide a seamless experience for future versions of the software. Once the user is able to introduce their own models, we would largely benefit from having more space to display all options that are available to add into the scene. On development, there was a concern that creating too many menu pages would hide away frequently used features. However, on testing this was found to have a minimal impact on the user’s work flow if the options were hidden one-click away and were clearly sign posted.

Users praised the simple layout of the system compared to that of AutoDesk3D. On approaching the professional software, many were confused by the wide array of tools available in the task bar. They were unable to identify most tools based on the icons alone, and some did not recognise the

terms used in the highlight labels. To contrast, the users found the icons in our project intuitive and representative of their function - this is largely due to the simplicity of the actions that are performed. Once more complex tools are added, we will ensure to keep these icons as accessible as possible and minimise the use of jargon so that complicated icons can be identified with little technical knowledge.

### 5.1.3 Recommended Features

A final survey gave an opportunity for potential user's to give feedback on any features that they think that the system is missing, and that would advance the potential use of the project. These have been added to the work queue and will be prioritised based on the frequency of the feedback.

Here we have listed the most common responses:

- Allowing the grouping of objects to edit them together, such as moving or deleting many objects in one go.
- Grouping objects to isolate interactions and customise how each groups interact.
- Naming objects and providing a simple menu to select objects based on the labels.
- Undo and redo recent changes.
- Auto-save changes.
- Add ambient music and sound effects.
- Previewing object changes before committing.
- A help or FAQ.
- An option to choose between world templates that introduce different terrains.
- Editing the UI and menus available.
- Toggle the placement grid across both camera views, and highlight the currently occupied positions.
- A global library of premade models to choose from.
- Welcome page displaying shortcuts keys available, and option to revisit this at any time.

### 5.1.4 Conclusions

Overall, users were found to quickly familiarise themselves with the tools available and could create a diorama of a world with little issue. While they would appreciate more customisation - through a wider range of models available - they enjoyed constructing their own worlds and it was a shared agreement that this provides a simple introduction into the professional modelling tools they had trialed.



## 5.2 Performance Testing

The following tests focus on the use-cases described in Section 3.5.3. This has been done with a mixture of both automated and manual tests, using the Chrome Developer Tool for performance analysis. This gives information on CPU and memory usage, network activity, and health of the frames.

The interactions from pointer are also recorded, however these have insignificant input delay and negligible processing time.

### 5.2.1 Launching the Application

[U1] The user can launch the application within the browser of their choice by navigating to the local host. This loads the environment containing an empty world block with the “day” lighting setting.

The initial launching of the application consist of four stages: loading the local server, running of the programming scripts, rendering the scene, and painting. The proportion of this execution is shown in Figure 5.2.

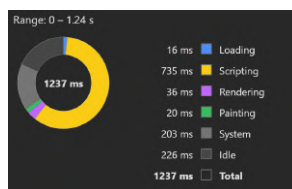


Figure 5.2: Stages of launching the application.

On the loading of the page, requests are sent over the network to connect to the local server and fetch the dependencies. This retrieves the separate modules of code that construct the individual features available, and the Three.js imports supplying the object loaders and camera controls.

Through the rendering phase, a render tree is constructed: this links the elements of the HTML DOM (Document Object Model) with their corresponding styles within the CSS StyleSheet [55]. Within this stage the flaticons used in the taskbar are loaded from the hosted server, so that the complete CSS tree can be constructed. The exact positioning is then calculated for each element, and they are painted onto the screen [55]. During the painting phase of the render, the images for each model are requested from the local server so that the menu can be constructed.

The assets to construct the starter environment are requested after the scripting stage, this consists of the first world block and the “day” environment mapping. The time to complete each of these requests is shown in the Network Activity chart in Figure 5.3.

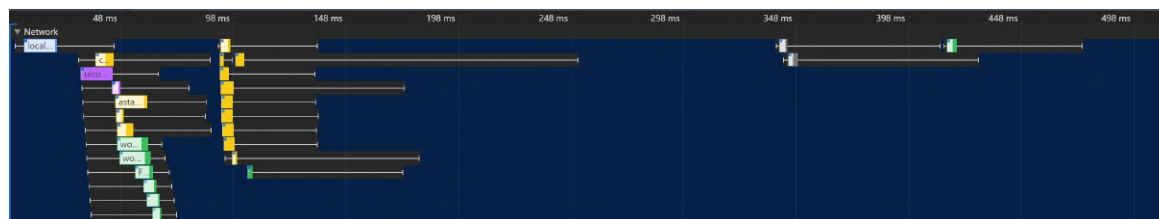


Figure 5.3: Chart showing the network activity when loading the application within a browser. The size of each component reflects the time for each request in milliseconds (ms), and they are coloured by the corresponding phase following the palette key given in Figure 5.2. This shows a clear wait between the loading of the key dependencies and the loading of the assets to begin the creation of the world.

The overall CPU usage can be seen in Figure 5.4, this is separated into the task stages given in the previous Figure 5.2. The broken blue bar shows the two separate stages of network activity, and the line graph represents the increase of memory usage throughout the loading process. This is due to the rise of assets being added into the browser. The red highlights at the top of the chart indicate that the maximum CPU usage recommended has been reached, warning that the users experience may be effected [31]. This is due to the corresponding long tasks that are being executed by the main thread.

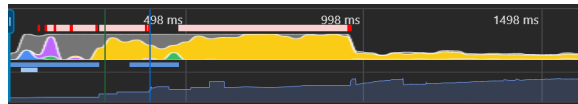


Figure 5.4: CPU usage for each stage when launching the application.

The Chrome Performance Insights have been used to analyse the source of these delays. The main cause of this performance bottleneck is found to be the animation loop which is called every frame, this is due to the repeated update of the camera controls and the rendering of the objects and lights. Another highlighted issue is the high computation necessary for the **RGBELoader** to parse the environment mapping assets. Techniques to optimise this are discussed in Section 5.5.

### 5.2.2 Adding Objects to the World

[U2] The user can select from the scenery icons within the menu and add 5 objects into the world.

The chart in Figure 5.5 shows the corresponding network requests to fetch each model from the local host, indicated by the bottom white lines. These requests have an average retrieval speed of 9.38ms, and have no significant impact on the frame-rate or CPU performance. This is due to the simplicity of the models, as being low-poly they have a reduced number of triangles for the GPU to parse. This meets our non-functional requirement R1 given in Section 3.1.

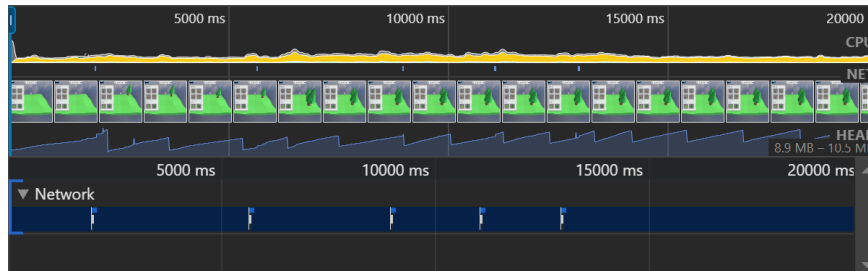


Figure 5.5: System responses when adding simple models to the scene.

### 5.2.3 Adding Characters to the World

[U3] The user can add a character into an empty world. The user then starts the movement and allows the character to complete its path, until it revisits the starting position. The user can then pause movement.

To contrast, the character models have a significant impact on the browser performance. The first interaction recorded in Figure 5.6, is in response to the user adding the character to the scene. Fetching the asset from the local sever has a significantly greater network response time than the simple static models, 91.79ms. This glTF model is much more complex, containing a higher number of triangles, vertices and textures, along with additional data such as animation clips and bones.

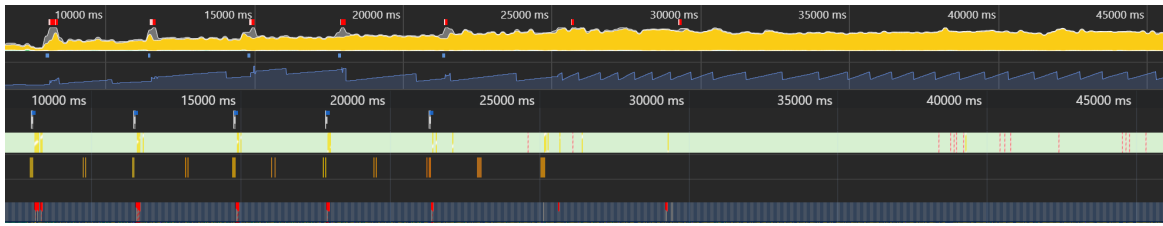


Figure 5.6: A chart showing the performance impacts on the addition of a character model. This visualises the peak in CPU computation, and partial decrease in frame rates.

This interaction is paired with a spike of CPU usage, and a brief period of partial and dropped frames. As shown in Figure 5.7, adding a character into the scene prompts a ‘long task’, this is the extended function that is triggered on the completed load of the glTF.

The chrome development tools classify this as a task taking longer than 50ms to execute, in this particular instance the task took 160ms, however this is dependent on available resources so varies.

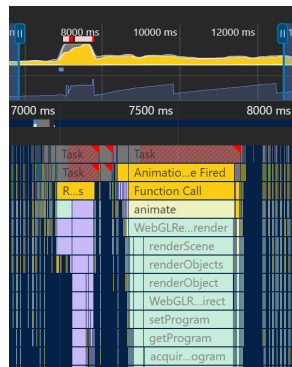


Figure 5.7: The main tasks called in response to adding a character to the scene.

This delay introduces latency since the browser must complete this task before it can respond to any of the user’s interactions, since the main thread can only work on one task at a time [31]. The process of adding characters into the scene has great potential of causing a bottleneck in performance, and can be optimised by breaking it up into sub-tasks.

For example, currently the paths for each character are generated as part of the introduction process, however this computation can be delayed until the user has decided to begin the movement. This avoids unnecessary resource use, as these paths may be generated and then immediately re-computed to represent an updated scene, or the user may never have use for this feature. This decreases the time allocated in response of the addition of a character, and instead leaves the browser open to respond to higher priority tasks, such as user interaction [31].

When the user indicates for movement to begin, there can be a “buffer” stage which indicates that the application is now busy computing the paths for the characters, and user interactions have temporarily decreased in priority. Once this computation has finished, the characters can begin their movement across the scene, and normal user interactions can resume.

### 5.2.4 Computing a Character's Path

[U4] The user can add an object into the scene and add a character. The user starts the movement and lets the path be completed as before, then pauses movement. The character should avoid this object.

This test analyses the performance difference associated with the addition of a static object that the character must take into consideration when generating its path. The results can be seen in Figure 5.8.

Here we see the same drop in frames caused by the addition of a character into the scene, but minimal impact based on the presence of a static object. This is due to the updated path-finding method discussed in Section 4.2.3, having negligible computation caused by pre-existing models. When instead considering the previous method, discussed in Section 4.2.2, the computation would increase exponentially with each object added.

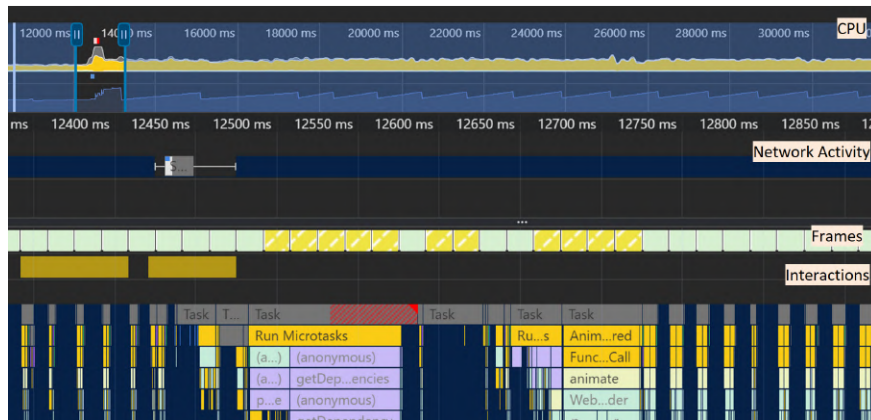


Figure 5.8: Chart showing the resource usage, frame health, and network activity, in response to adding a character to the scene and computing its path around an object.

### 5.2.5 Revising a Character's Path

[U5] The user can add character and play movement. They can then place an object onto its path, and the character should recompute its route to avoid this collision. Once the character has revisited its starting position, pause movement.

No significant issues are raised from this test, as the revision of the path has not caused any major impacts on performance. The frame-rate has remained within our allowed range, and there is no significant spikes in CPU usage. This can be seen in Figure 5.9.

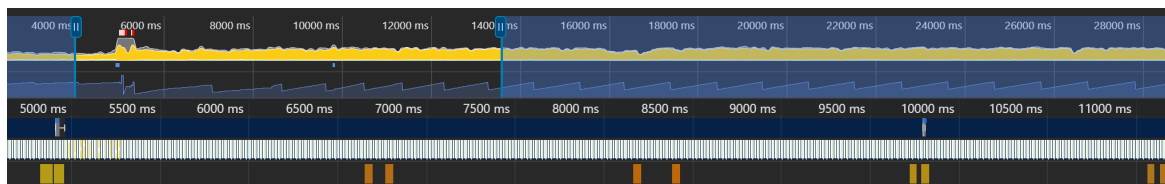


Figure 5.9: The resource usage when revising a characters path by deliberately causing a collision.

### 5.2.6 Adding Multiple Characters

[U6] The user can add 5 characters into the scene, and then play movement. The characters should all travel along their individual paths, interacting with each other to avoid any collisions that may take place.

The development tool results can be seen in Figure 5.10, this visualises peaks in CPU computation caused by the introduction of each character - indicated by the markers in the second row. Each addition causes a temporary drop in partial frames, however this is unobserved by the user.

Throughout the test there is a steady increase in CPU usage. Once the characters begin to explore the scene, indicated by the final marker in the ‘Interactivity’ row, occasional peaks occur in resource use. This is due to extra computation handling the avoidance of dynamic collisions. There are two instances of significant increase, highlighted in red, which may impact the user’s experience- this was noticed by a small lag on character movement. [chrome] The resource usage is dependent on the randomness in the paths generated and the number of interactions that will take place.

Several occurrences of dropped frames can be seen near the end of the test, this is likely due to the character interactions and rendering the complex models, however this decrease in frame rate is not excessive as was not noticeable by the user.

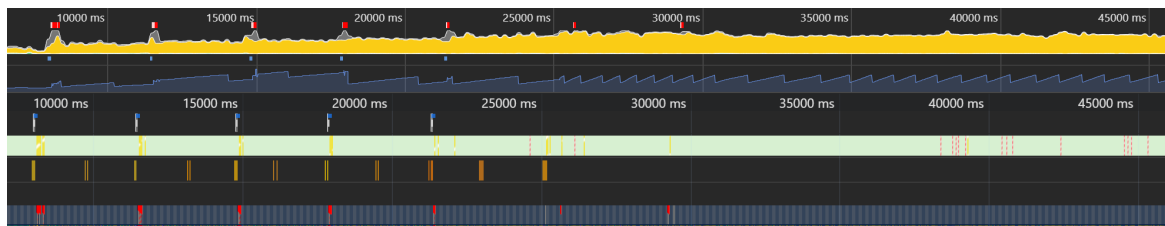


Figure 5.10: Resource usage and frame health when adding five characters into the scene and allowing them to interact amongst themselves.

## 5.3 Stress Testing

These stress tests are performed to find the limit of the system when increasing the complexity of the scene. The number of objects that can be added before the browser’s response time is effected, depends on the complexity of the objects. Due to the large difference in the complexity of the static and dynamic models, these are tested independently.

The Windows 3D visualiser has been used to explore the complexity of construction for each of the model’s mesh and texture sizes. These are given in Table 5.1, and are used to inform our performance tests and are taken into consideration when measuring the benchmark for the system. Any limitations to conserve performance are applied only once system response time begins to be effected, rather than being dependent strictly on number of models. This is as the user may add in their on models of differing complexity.

| Model     | Type    | Triangles | Vertices | Materials |
|-----------|---------|-----------|----------|-----------|
| Tree1     | Static  | 316       | 556      | 2         |
| Tree2     | Static  | 36        | 66       | 2         |
| WorldBase | Static  | 184       | 452      | 2         |
| NPC1      | Dynamic | 7900      | 15864    | 6         |
| NPC2      | Dynamic | 6424      | 12968    | 7         |

Table 5.1: Complexity of each model used in performance tests.

### 5.3.1 Maximum Models on a Block

[U7] Incrementally add static models to a single world block until there is a significant decline in performance.

Initially 20 simple object models are added into the world. Each recorded interaction from the pointer is paired with a network request, signifying that the user has clicked the menu to load a new model. The following interactions represent the pointer input choosing the position to place the model onto.

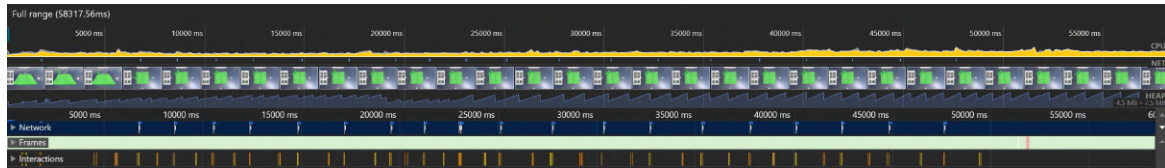


Figure 5.11: Resource usage when adding 20 simple models into the scene.

The chart in Figure 5.11 shows the gradual increase in CPU usage along with the increase in models being added. While a frame is dropped after 20 models being added, there is not yet any significant impacts on the user experience. There is an average presentation delay after each pointer interaction of 36.2ms, this is negligible and not noticed by the user.

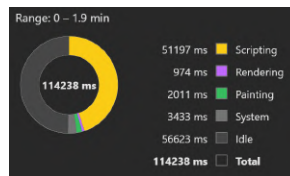


Figure 5.12: The time allocated to each stage of loading the web application after the addition of 40 simple models.

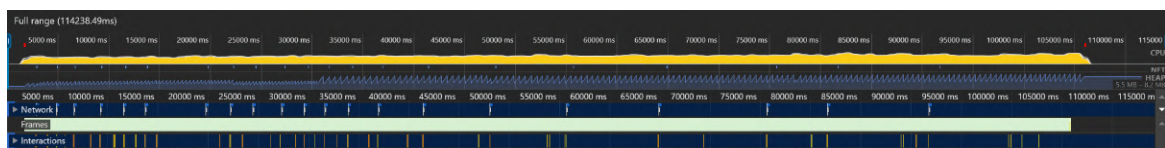


Figure 5.13: The resource usage when adding 40 simple models to the scene.

To test the limitation of the system, an additional 20 simple models are introduced in the following test stage. This results in a significant increase in scripting time, as shown in Figure 5.12. This has increased the average delay from fetching the models to 12.64ms, and the average delay following user interactions to 56.6ms. This quickly becomes noticeable to the user, experiencing a long delay before the chosen object is rendered into the scene to be translated.

Following this test, it has been decided to add a on the number of objects across a single world base, once response time becomes. For the current models within scope of the project, this is 30 static objects, though is dependent on the user's network and CPU resources available.



### 5.3.2 Extreme Use Case: Adding Objects

[U8] Expand the world significantly and add the maximum number of objects.

This test is repeated for an increasing number of world blocks until there is noticeable impact. Once 24 world blocks are added to the scene, the Three.js stats module indicates that the average frame rate reduces to 82fps compared to the initial 120fps. While this is within our requirements, there are the beginning of delays in response to user interactions, therefore a suitable limit to impose would be 30 world blocks, with more added only if the user's machine can respond without significant impact.

To test the limits of the system, 80 objects have been added on this expanded canvas. This is considering only the simple static objects. This world can be seen in Figure 5.14.

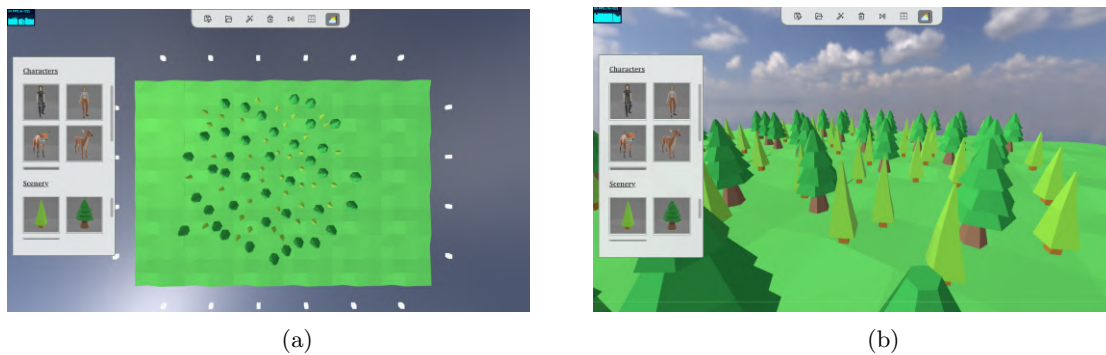


Figure 5.14: An example use case of 80 objects added to an extended world.



Figure 5.15: The resource usage and frame health when adding 80 models to the scene.

From Figure 5.15, we can see there are no long tasks in the main thread as we are only including simple objects, however the CPU usage steeply increases as more objects are added into the scene and eventually this maxes out.

Frames start being dropped on the introduction of scenery models, giving a total of 25 objects in the scene, and the CPU usage continues to increase for each new object addition. This reaches a peak after 40 scenery objects have been added - with a total 64 models in the scene.

Once more models are added after this, there is an increase in the frequency of frames being dropped or partially presented. The development tool has also identified several points that could have large impacts on the user's interactivity with the application. This is noticed during the tests as a large delay when loading a new object into the scene, and a lag when positioning the object into its place. The object is seen as dragging behind the pointer, rather than being stuck to it.

Due to the rasterisation of Three.js, only objects within the camera's viewpoint are rendered. This means there is noticeable latency only when the camera has been manipulated to view the entirety of the scene at once. At this point, frame rate drops to around 60fps.

From these tests we can add a limit of 60 objects into the scene - including both world blocks and scenery models.

### 5.3.3 Extreme Use Case: Adding Characters

[U9] Incrementally add character models to a single world block until there is a significant decline in performance.

For this 10 characters have been added to the starter world block, see Figure 5.16.



Figure 5.16: An example use case adding 10 characters into the scene.

As shown in Figure 5.17, each addition causes a spike in the CPU usage, reaching the recommend limits. Once the movement has been played and the characters begin interacting, the CPU usage reaches its max, causing significant delay in the user interactions within the browser. The dropped frames appear in groups, correlating with the later models being added, and the occurrences of frequent interactions.



Figure 5.17: The resource usage and frame health when adding 10 characters to the scene and allowing them to interact.

## 5.4 Optimisation Techniques

A limitation is placed on the number of complex characters that can be introduced to the world, without causing significant performance loss. This threshold is applied to optimise the system and avoid the risk of overloading weak clients, by limiting scene complexity. In the future, this needs limit to be responsive to the complexity of the characters that are being added into the scene.

The addition of characters into the scene requires a much greater increase in resources, compared to the introduction of the static scenery objects. When adding static objects to the scene, it is necessary to revise the paths of nearby characters to ensure that the pre-existing paths do not now collide with the new object. Therefore, limiting the number of characters that can be added will also reduce this necessary computation.

The overall complexity of the scene is also limited by reducing the number of polygons involved in the creation of the modules, this decreases the workload on the GPU processors when rendering the scene [56].



## 5.5 Future Performance Gains

If it is found to be necessary, future development can focus on to decreasing the limitations of the system, by implementing further optimisation techniques that were found in the initial research phase.

Alongside **Object3D** objects, Three.js also offers the use of **Level of Detail** objects (LOD). These are composed from multiple versions of the same mesh, each of a different quality level [57]. This allows the renderer to draw objects further from the camera with less detail [58].

While we are not often creating a scene with incredible depth, this could be an issue if the user extensively expands the world in a single direction, or zooms far out of the scene.

If the window is becoming less responsive, there is an increased lag between the user's action and the server's response, there are several approaches to reduce the number of assets loaded into the scene without overly restricting the user's freedom. The render distance of the camera can be reduced, or we can toggle the visibility of certain objects so that they are no longer rendered- even though they are within the visible range of the camera [57]. For example, small scenery objects that are far away from the camera can be hidden from the renderer, but larger objects or characters would continue to be displayed as these may be more significant for the user to view.

### 5.5.1 Design Methodologies

There are two main techniques commonly used to dynamically handle the optimisation of the system, through flexibility of resources available on the client's machine. These are popular design methodologies then ensure a web-page remains usable across different browsers, and has backwards compatibility of HTML versions available.

#### Progressive enhancement

This method begins with loading only the essential features, including a low threshold for world complexity - complex features are introduced based on the client's capabilities [59]. For example, using an older browser may only allow a small number of models to be added to the scene, and lacks additional enhancements such as shadows, anti-aliasing, or high pixel resolution [56]. These features would only be added if there is no significant impact on the performance, which may be represented by a drastic drop-in average frame rate, or a large increase in render time [26].

#### Graceful Degradation

To contrast, this approach begins with a high threshold for a complex world, and features are removed, or simplified, if they are found to hinder performance [59]. If rendering an object is causing an excessive buffer within the browser, then the complexity of the scene can be reduced by bounding the number of models that can be added to the scene or simplifying the rendered objects by removing the faces not visible by the camera [56].

## Chapter 6

# Project Management

This section discusses the main project management tools used when following the management methodology described in Section 2.1, and reflects on issues that arose.

### 6.1 Reflection

The initial plan is built from the schedule laid out in the Project Specification, this is updated as the project progresses, with any necessary amendments.

The creation of this took place before we had a detailed view of the technical steps required to meet our key objectives. This gave a broad plan of implementing all essential features within the first term, leaving the second term available for the development of extension features. Extra time was allocated for necessary research into the project's usability and key policies to follow to ensure development of an intuitive graphical interface. This included the conduction of the interactivity surveys, and the unstructured interviews with potential users within our target demographic.

Overall, this was plan followed, however some key adjustments were made. It was found that some interactivity features, such as the manipulation of models, benefit from early development. This is widely due to smoother testing and development of core features. For example, when implementing the interaction between characters, free placement of the models allowed for collisions to be forced without hard-coding meetings or relying on randomness.

A Gantt chart of the revised schedule is given in Figures 6.1 and 6.2, while the original schedule can be found in the Appendix C.

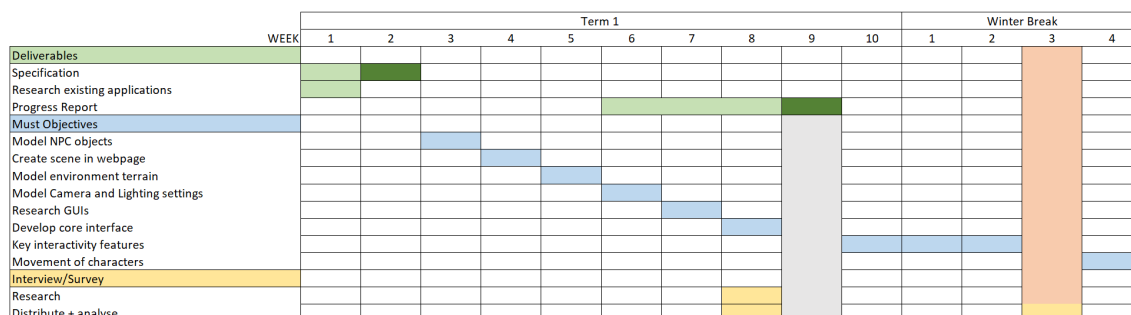


Figure 6.1: Gantt chart showing revised schedule for Term 1

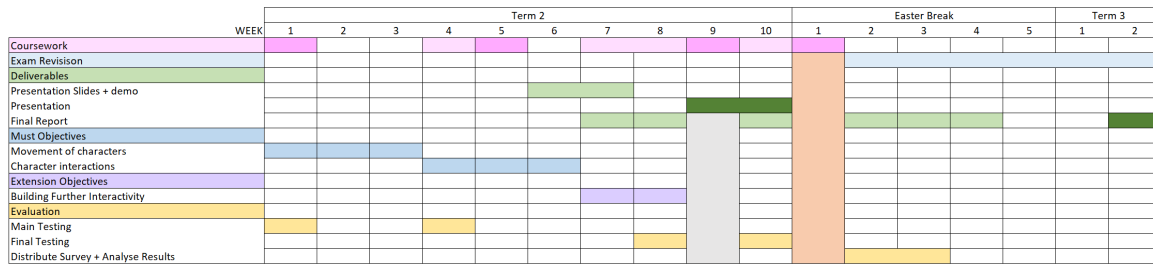


Figure 6.2: Gantt chart showing revised schedule for Term 2

## 6.2 Unforeseen Issues

As discussed, implementing the path-finding of the NPCs was postponed, and had a longer development period than initially anticipated – due to the additional implementation. During the risk assessment of this project, we had assigned contingency periods to each phase, and had significant time placed aside dedicated to implementing the extended features. Therefore, this risk was easily mitigated by reducing the number of low priority objectives finalised- this had limited repercussions as does not impact the Minimum Viable Product laid out in section 3.1.

Due to the rearrangement of the schedule, the basic interactivity features were implemented based on the tester models, these had no animations therefore interactions could be developed by casting rays from the users mouse position onto the models meshes. However, once movement and animations were introduced, the raycasting onto the final models was inaccurate due to their skinned meshes. This is since the position of the mesh is adjusted based on the animation in progress, hence accurate model selection requires significant additional computation. Since direct mesh selection is not within the projects scope, this issue has been avoided by producing a static selection box to follow the characters around the scene. All interactions are then done against this box, rather than the model themselves. This can easily be calculate as is built from a basic geometry.

The selection boxes must rotate along with the characters as they translate around their curved paths. An example of the created box is given in Figure 6.3, as the character performs it animations, there are often meshes that will leave this selection box. However, the system will respond to a users click on a large majority of the character, this accuracy is accepted for the current development.

Static objects, such as scenery pieces, are not affected by this and can continue to be computed as normal.

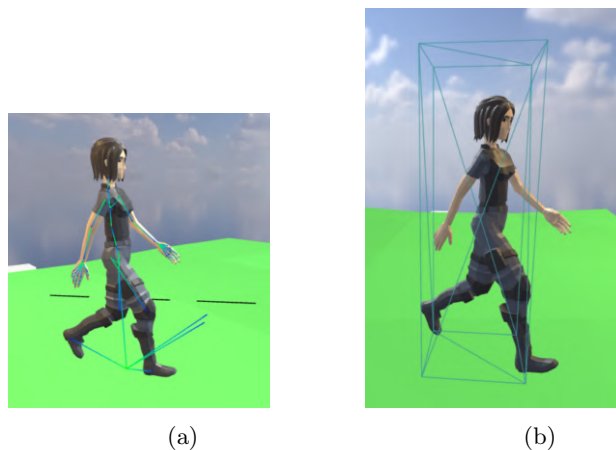


Figure 6.3: The skeleton of an animated model (a), and its created selection box (b).

## Chapter 7

# Evaluation

This section reflects on the progress made during the projects development, and evaluates the final product against our initial objectives. Here we discuss future development phases that would enhance the functionality of the system and applications past our current focused use-case.

### 7.1 Project Objectives

All core objectives have been achieved and have been implemented following the processes discussed in Section 4. Objectives with a lower priority have been fulfilled to a lower extent.

The ease of creating an animated and interactive scene, is a feature that other design tools do not accommodate.

This has been included in our project through the use of an A\* pathfinding algorithm to compute a route for characters to follow and explore the created world. This is implemented using a graphical representation of the scene constructed from the available spaces to move to. This ensures that the character will not be placed onto a co-ordinate that is currently occupied, or outside of the world boundary. A dynamic avoidance system ensures that NPCs will take steps to move out of the way of the other's path, once a character comes within range.

Adding an element of randomness to this movement is a low priority goal, and as such has only been partially implemented. This is through the use of a random number generator (RNG) to generate a random end point for the character to travel towards. Since the path is constructed following an A\* approach, there is no randomness in the steps that the character takes, instead following the most efficient route.

To strengthen this goal, an RNG can be used when moving the characters position. This would decide whether to move to continue following the created route, or to step away in a random direction. This decision could be made at random intervals, but it must be ensured that these are sufficiently far apart and not overly frequent that the path becomes unnatural.

Initially, it was planned to allocate movement speeds to each character based on their 'type'. Instead, our research phase found that the best approach to have dynamic objects is by updating their positions together within the rendering loop. Therefore, the speed of each character is determined by the length of their path generated. This is since each character will progress along their path based on a constant proportion value. The distance moved by the characters each interval is thus dependent on the overall path length.

## 7.2 Limitations

Development of the project focuses on the use case of creating a forest scene, therefore the user is restricted to only the provided models.

Since the application will be provided as open source, the user can modify the included models by directly editing the coded reference to the models location. Expanding the scope of the project to include other use-cases, as discussed in Section 7.4, strengthens our motivation to provide the user a stepping stone into computer aided design. As more functionalities are added to the system, it becomes increasingly likely that a user will find benefits that outweigh the dedication required to perform similar tasks within more complex software.

It is essential that the usability of the application continues to be the main motivation for the project. Despite an increase in functionality, the user interface must remain minimalist and provide clear access to the most frequently used features - such as camera manipulation and the addition of objects - so as to not disrupt the users workflow.

As discussed through the testing of the project in Section 5.3, another limitation of this approach is the number of objects that can be used to construct the world. This is highly dependent on the complexities of the objects introduced, therefore a limitation is added dynamically. The user is prevented from adding further complexity to the scene once a threshold performance has been breached.

## 7.3 Next Steps

This section discusses the remaining extension features that would be added upon future opportunity, and potential use cases that the project could explore.

### 7.3.1 Interactivity Features

Within the next development phase, focus will be on implementing the remaining interactivity features in order to create a more seamless experience.

#### Heads Up Display (HUD)

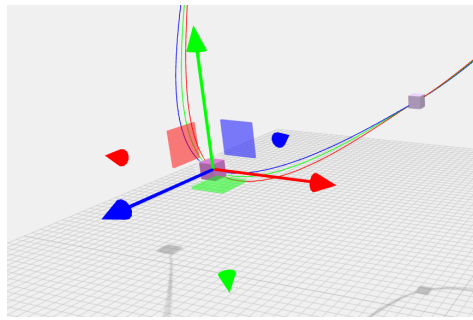
Placing an overlay across the selected object would help the identify the object in question and display the available actions to the user.

We have the option to include this through the use of the `TransformControls.js` addon, given in Figure 7.1a, however this gives a cluttered appearance very similar to that of Blender and gives a highly-technical experience which we are trying to avoid.

In order to keep the system as welcoming to beginners as possible, a more friendly interface may be appropriate. Such as the implementation in Planner5d. This is a room design software that indicates selection of objects through a HUD shown in Figure 7.1b. Including a feature in this style results in obvious access flows to the main customisation options available: such as rotate, duplicate, and delete. This maintains the casual appearance we are aiming for, and removes the requirement for the user to memorise keyboard shortcuts or navigate menus to find commands.

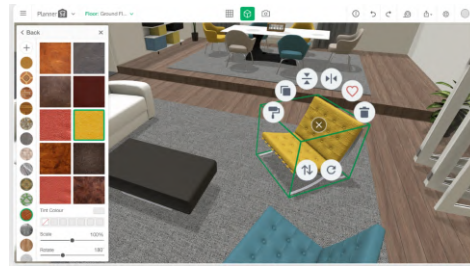
#### Importing models

This will remove the systems main limitation and allow for significant further customisation of the scene. This advances the project past the focused use case, and allows for the next significant development phases to take place, bringing the project closer to the future applications discussed in Section 7.4.



Source: [https://threejs.org/examples/?q=spline#webgl\\_geometry\\_spline\\_editor](https://threejs.org/examples/?q=spline#webgl_geometry_spline_editor)

(a) The Three.js addon TransformControls.js



Source: <https://planner5d.com/use/room-planner-tool>

(b) HUD showing interactions available in Planner5d.com

Figure 7.1: Different approaches to identify the selected object.

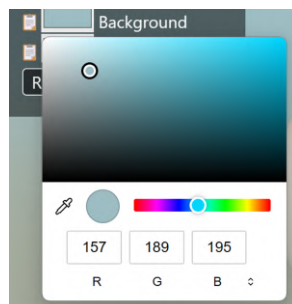
### Save and load worlds

Three.js allows the exportation of all models in the scene as a unified glTF model. This can be implemented in response to the ‘Save world’ selection, and a file explorer would allow the user to choose the location for this to be saved. Once the development phase introducing the import of models has been completed, a similar approach can be used to import the saved world back into the scene. Here, we can take advantage of the models hierarchy system to decompose it into the original objects to allow further customisation.

### Object Customisation

The current system recognises the user’s click onto an object’s mesh, and can update the colour of the material used through a programming script. Adding this feature into the graphical interface, as a separate page in the menu, will allow additional customisation of the scene, on par with having the user introduce their own models. By widening the functionality of the system beyond the project’s current scope, we will increase the assistance it can bring to new users and strengthen our main motivation.

This tool can be implemented through the use of the Three.js addon **three-colors**, which creates a graphical popup menu, as shown in Figure 7.2. This prompts the user to select their chosen RGB value through the use of a colour picker. It could be said that this approach is more intuitive than the colour sliders used in the Windows 3D Viewer, discussed in Section 1.2.1, as a novice with no graphical experience may not be familiar with allocating values to the separate colour channels.



Source: <https://three-colors.opl.io>

Figure 7.2: The addon three-colors

### 7.3.2 Technical Features

Additional technical features could be added in order to increase the user's overall experience of the system. This includes adding more customisation options as well as features that allow the user to have further interactions with the scene. This includes particle system to introduce weather, and additional small animations to play in response to the user's mouse click – such as the rustle of leaves in trees or more varied character responses.

This brings more game-oriented features into the software, and steps the project further away from a high-technical design tool. Our user testing found that beginner users appreciated this direction and believed that a casual application was more approachable than feature-heavy software.

#### Terrain Customisation

We can increase the use-case of our project by giving the user freedom to adjust the height of the terrain, and add features such as large mountains or lakes. This can be done with a drag-drop tool to allow direct modification of the world block. Alternately, the user could be presented with several world models to select which terrain type should be added as a world expansion.

#### Path Adjustments

The introduction of adjustable character paths adds further enhancements to the user experience - this allows for further customisation and brings use in more specific projects. The existing verbose mode toggles the display of the character's paths and the anchor points used. The user will be able to drag these points in order to directly manipulate the path. The new co-ordinate selected would be used as a way-point within the A\* algorithm, to ensure that this node is visited on a character's exploration.

## 7.4 Future Applications

While this project is developed with a focus on the use case of building up a dynamic world, there is a limitless potential for the applications this system could be expanded to provide. In this section a few examples are discussed:

### 7.4.1 3D Modelling and Visualisation

By allowing the user to import their own models into the scene, there is significant potential for the project to widen its target audience to include independent businesses who would like to model their products before production, but do not have any technical experience. This would be encouraged by additional customisation features, that allow the user to easily modify the scale and texture of components, without needing to navigate the endless technical features available in more specialised software.

Modifying the available models that can be introduced into the scene allows visualisation of an endless range of applications. This is beneficial for both creative and technical use cases. The user is able to visualise different spaces, such as a virtual office for remote work. Static objects can be replaced with furniture and wall objects, that can be positioned using the top camera to create an office layout and build up private rooms. Hosting the application online allows for the addition of multiple player-controlled characters that can be used to represent each worker in the office.

Kumospace is a video conferencing tool that allows the positioning of employee cameras to simulate an office environment, as shown in Figure 7.3. This employs a spatial audio system, that allows users to communicate when within detectable range [60]. If we were to expand the scope of our project to include use-cases such as these, then a similar feature can be added through the use of our existing collision detection.



Source: <https://www.kumospace.com/virtual-office>

Figure 7.3: Video conferencing with kumospace.

### 7.4.2 Game Creation

A player-controlled character can be implemented by reusing a significant amount of pre-existing code. More key-board event listeners must be added to allow for user defined movement, rather than following pre-set paths, and there must be a responsive collision avoidance system. The addition of this asset would bring the functionality of the system closer to a game developer.

The addition of this creates a “sandbox” environment for the user, allowing them to build up self-contained levels for their character to explore. This expands the system’s functionality to allow the creation of an open-world game using combinations of the supplied function blocks.

Without a controllable character, additional features will allow the user to create a simple idle game - such as introducing more interactions with the NPCs. Adding dialog to be triggered once the player approaches or selects an NPC, adds this extra level of game play that brings the system closer to that of a game designer. This can be applied using our existing object selection system.

An example of a game genre created through little modifications to the system is *Tsuki Odyssey* (2021) [61]. This mobile game allows the user to decorate a home through the use of a simple drag-and-drop system, similar to that implemented in our project. The character is not controllable by the user, but can be seen interacting with various objects through small animations when the user loads the game. The user is able to navigate the scene with a panning camera, and select NPCs to prompt small dialogue options.



Source: <https://hyperbeard.com/game/tsukisodyssey/>



Source: <https://hyperbeard.com/game/tsukisodyssey/>

Figure 7.4: Example images from the mobile game Tsuki Odyssey 2021

Our project abstracts the programming required for the creation of games similar to this, making computer graphics more approachable to those with minimal experience.

While this study obviously has other complexities, such as keeping a characters inventory and managing a shop system, its core features can be developed using the function blocks provided in this project. This focuses on other features, for example, our path-finding system would allow the characters to autonomously explore the world, and interact with the various objects in real time.



### 7.4.3 Scene Creation

Once the user has generated their customised scene, this can be used as a dynamic background for animations and games, or post-processed through a neural network to generate a photo-realistic world that reflects their design choices. This can be used as a source for image manipulation, such as splicing an image over the top of the background, or using the generated image to expand a photo.

Currently there exists a wide range of software that allows the user to generate images from a text prompt, and applications like ‘openart.ai’ allows the generation from a sketch [62]. The user can choose between different image styles, including photo-realistic and digital art, and provide example images to train the model. This produces a fleshed out image inspired by their sketched drawing. While the result of this is not always perfect, it shows the incredible applications that AI can have on digital creations.

An example of this software is shown in Figure 7.5, the given image has been produced from three iterations of the model.

By expanding our project, the user would be able to precisely construct how they would like their image to be created, through the individual positioning of placeholder models. A similar system will be used to then replace each model with a photo-realistic image, to generate a fully converted scene.

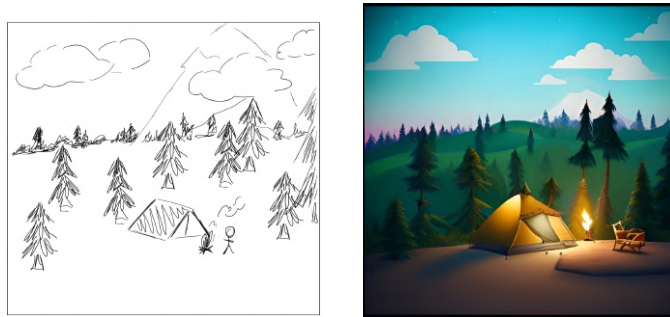


Figure 7.5: An example image generated using openart.ai, and the sketch used as a prompt.

### 7.4.4 Uses in Digital Art

Modelling scenes in an environment like Blender is commonly used as a helper tool for drawing backgrounds from complex perspectives, often beneficial for animating hand drawn shows. Software such as the one produced in this project, allows the user to create a scene quickly and intuitively, without the painstaking process of learning modelling or programming skills. The freely positional camera, with limitless zoom and rotate capabilities, allows the user to take a snapshot of the scene from the desired perspective, which can then be drawn over.

## 7.5 Conclusion

Computer graphics is a rapidly advancing domain that only 30 years ago was restricted to the rendering within a 2D plane with hand drawn perspective. Now, we are able to manipulate these models within a three-dimensional environment - observing the shape from all possible orientations, and precisely alter each individual mesh and vertex.

Over the course of this project, we have developed a system that provides a gentle introduction into computer graphics. While this has less sophisticated tools than the graphical software available, the lack of technical features is sacrificed for a more welcoming environment. Our project works on a higher level to the software currently available: rather than constructing models from scratch we focus on the positioning of premade objects to build up a diorama of a world.

This project expands on the abstraction onto WebGL that is provided by Three.js, allowing it to be executed within the user's browser. The purely graphical interface creates a visual and natural approach towards customising the world, an interactive world, which can be freely explored and viewed from any position; while encapsulating the scripts necessary to place and manipulate these models. The source code of this project provides function blocks that can be called within a users own project. This reduces the technical knowledge usually required, allowing the user to create a scene without the understanding of concepts such as meshes, geometries or materials. While these aspects are an important part of computer graphics, the user is able to “opt-in” to this, and is often unnecessary for many.

Keyboard shortcuts have been used to help streamline the world creation process, allowing for rapid prototyping. The analysis in Section 1.2.1, on Blender's extensive use of hot keys, has been taken into consideration and it has been decided that this accelerator is often highly desired by users. This removes the need to search for frequently used tools, and reduces menu pages which clutter the screen. As our research highlighted this as a key weakness in professional systems, steps have been taken to ensure that usability is maintained. All important features can also be accessed through the interface, so that memorising the shortcuts is not a necessity, and the mapped keys are kept as intuitive as possible: this has been ensured by building from conventions and user expectations taken from our survey results.

Compared to other tools, which allow for precise modifications in vertex and edge position, the produced system allows only edits on the highest level. This simplification allows the user full customisation when positioning objects, and brings the focus onto basic techniques involved in creating a scene.

Interview participants have expressed their needs for a simple modelling environment, these uses include, for example: a video export could be used as moving backgrounds for computer animation, while static captures can be used alongside traditional drawing techniques, or as the basis for generating a photo-realistic scene. Many users do not require the complex features included in professional software, and are overwhelmed by the abundance of tools available that are not necessary for their simple and casual goals. The needs of hobbyists are often overlooked by 3D modelling software, excluding users who do not have the time and patience required to learn the complexities of this skill.

# Acknowledgements

I would like to thank for my supervisor Dr. Rossella Suma, for her invaluable patience and feedback. I am immensely grateful for her guidance throughout this project, which has undoubtedly saved me time and headaches. I also extend my thanks towards Professor Till Bretschneider, whose insightful questions have prompted discussions within this report.

I give my deepest gratitude to my friends and family who have kept my spirits high, and to my cohorts who have provided extensive feedback and advice. Lastly, I thank my rubber duck and his unwavering support.

# Bibliography

- [1] Blender Team and Community. *Blender 4.1 Manual - Getting Started: About Blender*. Blender, 2024.
- [2] Mehrafrooz B, and Aminian P. *The Animator’s Playbook: Essential Animation Styles Explored*. <https://pixune.com/blog/animation-styles/>. 2024.
- [3] Whited B. Daniels E. Kaschalk M. Osborne P. and Odermatt K. “Computer-assisted animation of line and paint in Disney’s Paperman”. In: (Aug. 2012). DOI: 10.1145/2343045.2343071.
- [4] Blender. *Get Involved - User Stories: More than stories*. <https://www.blender.org/get-involved/user-stories/>. 2024.
- [5] R. Baruah. *Diving into Three.js*. In: *AR and VR Using the WebXR API*. Apress Berkeley, CA, 2021.
- [6] C. Shneiderman B. & Plaisant. *Designing the user interface: strategies for effective human-computer interaction, Fifth edn*. Boston: Addison-Wesley, 2010.
- [7] S.P. Snodgrass and A. Kontostathis. “Building a 3D Game in Multiple Environments”. In: *The International Journal of Visual Design* 6.4 (2013), pp. 45–54.
- [8] Unity Learn. *Learn game development w/ Unity — Courses & tutorials in game design, VR, AR & Real-time 3D*. <https://learn.unity.com/>. 2024.
- [9] Penny de Byl. *Finite State Machines*. <https://learn.unity.com/project/finite-state-machines-1>. 2023.
- [10] Unity Technologies. *Patrolling the Perimeter*. <https://learn.unity.com/tutorial/patrolling-the-perimeter#>. 2022.
- [11] Unity Documentation. *Navigation System in Unity*. <https://docs.unity3d.com/2021.3/Documentation/Manual/nav-NavigationSystem.html>. 2024.
- [12] Unity Documentation. *Nav Mesh Obstacle*. <https://docs.unity3d.com/2021.3/Documentation/Manual/class-NavMeshObstacle.html>. 2024.
- [13] Castano R. Dotto G. Suma R. Martina A. and Bottino A. “VIRTUAL-ME: A Library for Smart Autonomous Agents in Multiple Virtual Environments”. In: *Agent Technology for Intelligent Mobile Services and Smart Societies*. Communications in Computer and Information Science, vol 498. Berlin, Heidelberg: Springer, 2015, pp. 34–45.
- [14] Project Management Institute. *The standard for project management and a guide to the project management body of knowledge (PMBOK guide), Seventh edn*. Newtown Square, Pennsylvania: Project Management Institute, Inc, 2021.
- [15] R Erne. “How Can Projects Be Managed in a ‘Lean’ Way?” In: *Lean Project Management - How to Apply Lean Thinking to Project Management*. Springer, Wiesbaden, 2022.
- [16] Lurie M de Smet A and St George A. *Leading agile transformation: The new capabilities leaders need to build 21st-century organizations*. McKinsey & Company, 2018.
- [17] CG Cobb. *The Project Manager’s Guide to Mastering Agile : Principles and Practices for an Adaptive Approach*. Incorporated, Newark: John Wiley & Sons, 2015.
- [18] Quaternius. *Ultimate Animated Animal Pack*. <https://quaternius.com/packs/ultimateanimatedanimals.html>. 2021.

- [19] Quaternius. *Ultimate Modular Women Pack*. <https://quaternius.com/packs/ultimatemodularwomen.html>. 2022.
- [20] HDRI-Haven. *About HDRI-Haven*. URL: <https://hdri-haven.com/about>.
- [21] polyhaven. *Asset License*. URL: <https://polyhaven.com/license>.
- [22] Unity. *Performance profiling tips for game developers*. <https://unity.com/how-to/best-practices-for-profiling-game-performance>. 2024.
- [23] Archbold J. *CS261 Software Engineering : Human Computer Interaction*. University of Warwick Lecture Slides. 2023.
- [24] Philip G.C. “Software design guidelines for event-driven programming”. In: *Journal of Systems and Software* 41.2 (1998), pp. 79–91.
- [25] M Fowler. *Refactoring: improving the design of existing code*. Reading, MA.: Addison-Wesley, 1999.
- [26] J Dirksen. *Learning Three.js - the JavaScript 3D Library for WebGL : Create Stunning 3D Graphics in Your Browser Using the Three.js JavaScript Library*. Birmingham: Packt Publishing, Limited, 2021.
- [27] IBM. *What is software testing?* <https://www.ibm.com/topics/software-testing>. 2024.
- [28] I Sommerville. *Software Engineering, Global Edition*. Harlow: Pearson Education, Limited, 2015.
- [29] testomat.io - Sheremeta, O. *Playwright Testing Tutorial on How to Organize an Advanced (Scaled) (e2e/unit) Testing Framework*. <https://testomat.io/blog/playwright-testing-tutorial-on-how-to-organize-an-advanced-scaled-e2e-unit-testing-framework/>. 2022.
- [30] P. Leloudas. “Software Testing Types and Techniques”. In: *Introduction to Software Testing*. Berkeley, CA: Apress, 2023.
- [31] Basques, K. *Analyze runtime performance*. <https://developer.chrome.com/docs/devtools/performance>. 2017.
- [32] Bruegge B & Dutoit A. *Object-Oriented Software Engineering Using UML, Patterns, and Java : Pearson New International Edition*. Harlow: Pearson Education, Limited, 2013.
- [33] three.js Manual. *Loading 3D models*. <https://threejs.org/docs/#manual/en/introduction/Loading-3D-models>. 2024.
- [34] Joey de Vries. *Learn OpenGL - Graphics Programming: Learn modern OpenGL graphics programming in a step-by-step fashion*. Kendall & Welling, 2020.
- [35] three.js Manual. *GLTFLoader*. <https://threejs.org/docs/index.html#examples/en/loaders/GLTFLoader>. 2024.
- [36] three.js Manual. *WebGLRenderer*. <https://threejs.org/docs/?q=Renderer#api/en/renderers/WebGLRenderer>. 2024.
- [37] Raza S. *CS234 Computer Graphics : Shaders and Interactivity*. University of Warwick Lab Notes. 2024.
- [38] three.js Manual. *OrbitControls*. <https://threejs.org/docs/#examples/en/controls/OrbitControls>. 2024.
- [39] three.js Manual. *Backgrounds and Skyboxes*. <https://threejs.org/manual/?q=.exr#en/backgrounds>. 2024.
- [40] Raza S. *CS234 Computer Graphics : Lighting and Shading*. University of Warwick Lab Notes. 2024.
- [41] Haines E. & Hoffman N. Möller T. *Real-time rendering, 4th edn*. Milton: Chapman and Hall/CRC, 2018.
- [42] Blender. *Rendering and Beyond*. <https://www.blender.org/features/rendering/>. 2024.
- [43] Raza S. *CS234 Computer Graphics : Ray Tracing & Radiosity*. University of Warwick Lecture Notes. 2024.

- [44] three.js Manual. *Box3*. <https://threejs.org/docs/#api/en/math/Box3>. 2024.
- [45] Grinstead, B. *A\* Search Algorithm in JavaScript (Updated)*. <https://briangrinstead.com/blog/astar-search-algorithm-in-javascript-updated/>. 2010.
- [46] rosettacode. *Map range*. [https://rosettacode.org/wiki/Map\\_range](https://rosettacode.org/wiki/Map_range). 2024.
- [47] three.js Manual. *Animation System*. <https://threejs.org/docs/#manual/en/introduction/Animation-system>. 2024.
- [48] three.js Manual. *AnimationMixer*. <https://threejs.org/docs/#api/en/animation/AnimationMixer>. 2024.
- [49] three.js Manual. *Raycaster*. <https://threejs.org/docs/?q=ray#api/en/core/Raycaster>. 2024.
- [50] Reynolds C.W. “Flocks, herds and schools: A distributed behavioral model”. In: *ACM SIGGRAPH Computer Graphics* 21.4 (1987), pp. 25–34.
- [51] Columbia University. *How to work with Planes in three.js*. <https://www.columbia.edu/~njn2118/journal/2019/2/18.html>. 2019.
- [52] three.js Manual. *Ray*. <https://threejs.org/docs/?q=ray#api/en/math/Ray>. 2024.
- [53] Three.js University. *Chapter 6 – Sprites and Particles in Three.js*. <https://en.threejs-university.com/2021/08/03/chapter-7-sprites-and-particles-in-three-js/>. 2024.
- [54] three.js Manual. *Object3D - Sprite*. <https://threejs.org/docs/index.html?q=spr#api/en/objects/Sprite>. 2024.
- [55] Shekhar, S. *Complete Webpage Rendering Process In Browser*. <https://dev.to/shwetabh1/complete-webpage-rendering-process-in-browser-5911>. 2021.
- [56] Three.js University. *Optimizing a Three.js application – Tips for achieving a fluid rendering at 60 FPS*. <https://en.threejs-university.com/2021/08/06/optimizing-a-three-js-application-tips-for-achieving-a-fluid-rendering-at-60-fps/>. 2024.
- [57] Pfister, D. *Level Of Detail Objects in three.js*. <https://dustinpffister.github.io/2023/05/19/threejs-lod/>. 2023.
- [58] three.js Manual. *Object3D - LOD*. <https://threejs.org/docs/#api/en/objects/LOD>. 2024.
- [59] V. Gor. “Cross-Browser Challenges and How to Resolve Them.” In: *Creating Responsive Websites Using HTML5 and CSS3*. Berkeley, CA: Apress, 2023.
- [60] kumospace.com. *Virtual Office*. <https://www.kumospace.com/virtual-office>. 2024.
- [61] Rapbot Studios. *Tsuki Odyssey v1.9.16*. <https://hyperbeard.com/game/tsukisodyssey/>. Mexico City, Mexico, 2021.
- [62] openart.ai. *Sketch To Image*. <https://openart.ai/apps/sketch-to-image>. 2024.

## Appendix A

# Dynamic Collision Logic

Listing A.1: Detecting Collisions

```
1
2 # detect a potential collisions within the range a character can see
3 Function detectCollision():
4     let facing = npc.getWorldDirection()
5     #Three.js method, returns a vector representing the characters
6     # local positive z-axis
7     Send forward ray from npc position in the diection of facing
8
9     const angle =  $\theta$ 
10    let left = facing.applyAxisAngle( y-axis , angle)
11    let right = facing.applyAxisAngle( y-axis , -angle)
12    Send rays from npc position in direction of left and right
13
14    let others = list of characters in the scene minus the current npc
15    let interactions = rays.intersectObjects( others )
16    # Three.js method, retruns list of intersections with the given array
17
18    If (intersections) then
19        Get character name from intersections
20        Return collided character
21    End If
22    Else
23        Return False
24    End If
25 End Function
```

Listing A.2: Avoiding Collisions

```
1
2 # Avoid a potential collision by pausing path progression
3 # and adjusting the characters path
4 function characterInteraction(npc1, npc2):
5     # Rotate characters to face eachother
6     # 1. Get direction vector between them
7     let pos1, pos2 = position of each npc
8     let direction = pos1 - pos2
9
10    # 2. Calculate the equivalent opposite vector
11    let opposite = (-direction.x, direction.y, -direction.z)
12
13    # 3. Rotate npcs to face along these vectors
14    npc1.lookAt( direction + pos1 )
15    npc2.lookAt( opposite + pos2 )
```

```
16
17     Pause the characters paths
18     Get the animation mixer for both characters
19     Change the active animation
20
21     # Stager the movement of characters to avoid the collision
22     setTimeout ( duration of animation clip){
23         unpause npc1.path
24         adjust characters path to avoid the colliding character
25     }
26
27     setTimeout ( duration of clip *2){
28         unpause npc2.path
29     }
```



## Appendix B

# Survey Responses

Feedback from the first survey is shown in Tables B.1 and B.2.

|                    | Left Click | Right Click | Keyboard Shortcut | WASD Keys | Space | Toolbar | Mouse Movement | Mouse Scroll |
|--------------------|------------|-------------|-------------------|-----------|-------|---------|----------------|--------------|
| Remove object      | 0.00       | 34.78       | 47.83             | 0.00      | 0.00  | 17.39   | 0.00           | 0.00         |
| Remove all objects | 4.35       | 8.70        | 34.78             | 0.00      | 0.00  | 52.17   | 0.00           | 0.00         |
| Rotate object      | 0.00       | 21.74       | 8.70              | 13.04     | 4.35  | 26.09   | 21.74          | 4.35         |
| Duplicate object   | 0.00       | 26.09       | 52.17             | 0.00      | 0.00  | 17.39   | 4.35           | 0.00         |
| Customise colour   | 0.00       | 30.43       | 4.35              | 0.00      | 0.00  | 56.52   | 8.70           | 0.00         |
| Randomise scene    | 0.00       | 0.00        | 21.74             | 0.00      | 4.35  | 69.57   | 4.35           | 0.00         |
| Pan camera         | 4.35       | 8.70        | 0.00              | 21.74     | 0.00  | 8.70    | 47.83          | 8.70         |
| Zoom camera        | 0.00       | 0.00        | 4.35              | 0.00      | 4.35  | 4.35    | 13.04          | 73.91        |
| Pause movement     | 18.18      | 4.55        | 9.09              | 0.00      | 40.91 | 22.73   | 4.55           | 0.00         |

Table B.1: Results from survey showing how sample users would expect to use key features. Values given are the total percentage of results for this choice, and the most chosen option is highlighted.

| Feature             | Score | Percentage of total |
|---------------------|-------|---------------------|
| Free Placement      | 85    | 12                  |
| Save and Load Files | 85    | 12                  |
| Delete Object       | 81    | 11.4                |
| Rotate Object       | 75    | 10.6                |
| Grid Snapping       | 62    | 8.7                 |
| Import Objects      | 59    | 8.3                 |
| Duplicate Object    | 51    | 7.2                 |
| Canvas Expansion    | 51    | 7.2                 |
| Customise Colours   | 51    | 7.2                 |
| Display Grid        | 42    | 5.9                 |
| Randomise Scene     | 35    | 4.9                 |
| Remove all Objects  | 33    | 4.6                 |

Table B.2: Results from survey ranking features in order of importance to a user.

## Appendix C

# Original Project Schedule

The schedule developed during the inital planning stage is given in Figures C.1 and C.2.

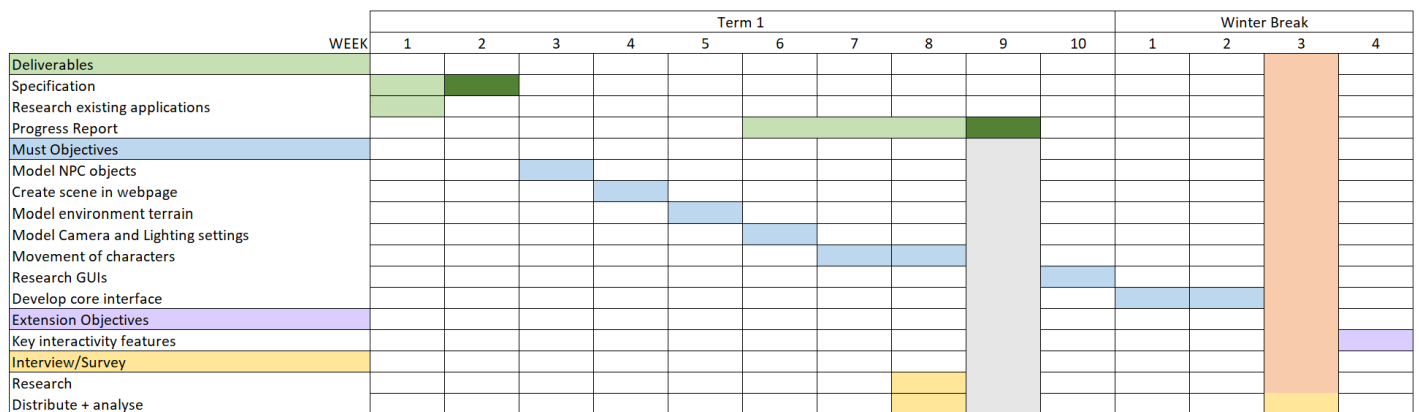


Figure C.1: Gantt chart showing original schedule for Term 1

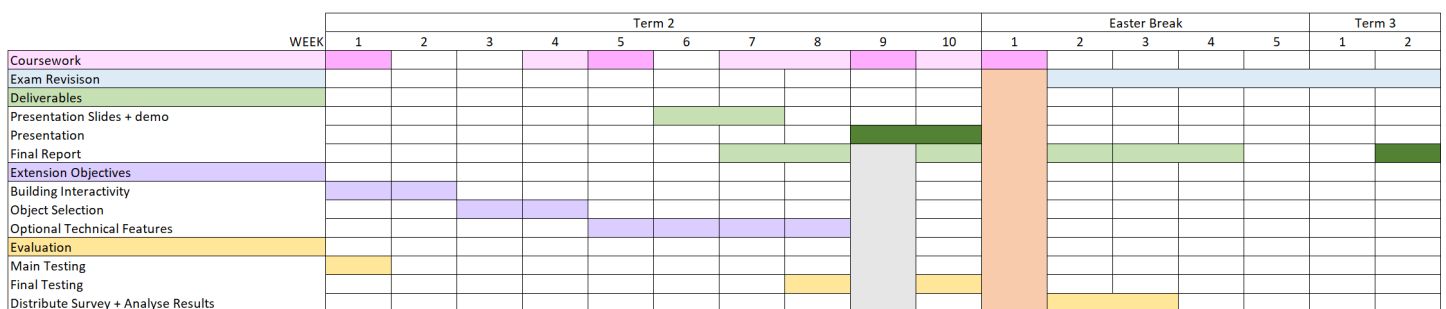


Figure C.2: Gantt chart showing original schedule for Term 2